

Modbench

User manual

A Modbus client for Windows, Linux and macOS: manual requests, device maps, charts, a simulator and tools to find and check devices.

Version 0.5.2 · November 2026
github.com/nobrega8/modbench

Contents

1 Introduction **1**

1.1 What you can do with it . . . 1

1.2 A few Modbus words . . . 2

1.3 The window . . . 3

2 Installation and updates **4**

2.1 Windows . . . 4

2.2 macOS . . . 4

2.3 Linux . . . 4

2.4 Updates . . . 5

2.5 Files Modbench opens . . . 5

3 The Main page **6**

3.1 Step 1: Connection . . . 6

3.2 Step 2: Request . . . 7

3.3 Address notation . . . 8

3.4 Step 3: Test . . . 9

3.5 Finding devices . . . 10

4 Device maps **11**

4.1 Opening a map . . . 11

4.2 The Device map tab . . . 12

4.3 Is it the right map? . . . 13

4.4 Checking a device against its map . . . 13

4.5 Map status . . . 13

4.6 Byte order override . . . 14

4.7 Computed values . . . 14

4.8 Exporting and recording . . . 14

4.9 What is remembered per device . . . 15

5 Reading and writing **16**

5.1 The Read registers tab . . . 16

5.2 Snapshots . . . 17

6 Charts **19**

6.1 Adding lines . . . 19

6.2 Reading the chart . . . 19

6.3 Markers . . . 20

6.4 Saving a chart as an image . . . 20

7 Raw data and the log **22**

7.1 The Raw data tab . . . 22

7.2 The Log tab . . . 23

8 The device library **24**

8.1 Your library . . . 24

8.2 Adding maps . . . 25

8.3 The map builder . . . 25

8.4 The online library . . . 27

8.5 Finding the map for a device . . . 27

9 The simulator **29**

9.1 Where it listens . . . 29

9.2 Devices . . . 30

9.3 Values . . . 30

9.4 Generators . . . 31

9.5 Its own window . . . 31

9.6 Saving a setup . . . 31

9.7 The demo . . . 32

10 Tools **33**

10.1 Identify . . . 33

10.2 Find addresses . . . 34

10.3 Network scan . . . 35

10.4 Exporting scan results . . . 36

10.5 Gateway . . . 36

10.6 Bus monitor . . . 36

10.7 Send request . . . 37

10.8 Data decoder . . . 37
10.9 Frame decoder . . . 38
10.10 Trend player . . . 38
10.11 Test sequence . . . 39
10.12 Statistics . . . 40
10.13 Alarms . . . 41

11 Settings **42**
11.1 About . . . 42
11.2 Appearance . . . 43
11.3 Values . . . 43
11.4 Device library . . . 43
11.5 Updates . . . 44
11.6 Connection . . . 44
11.7 What Modbench keeps . . . 44

12 Command line and AI assistants **45**
12.1 Commands . . . 45
12.2 Generators file . . . 47
12.3 AI assistants (MCP) . . . 47

A Troubleshooting **49**
A.1 No answer (timeouts) . . . 49
A.2 Values look wrong . . . 49
A.3 Bad frames . . . 49
A.4 Exception codes . . . 50

B Reference **51**
B.1 Function codes . . . 51
B.2 Byte and word order . . . 51
B.3 Files . . . 51
B.4 Keyboard . . . 52

CHAPTER 1

Introduction

Modbench is a Modbus client: it sends requests to Modbus devices (meters, inverters, drives, PLCs, controllers) and shows what they answer. It reads and writes registers by hand, or by name with a *device map*, charts values over time, shows every frame on the wire, and simulates devices when there is no hardware at hand.

It runs on Windows, Linux and macOS, speaks Modbus over serial lines (RTU and ASCII) and over the network (TCP, UDP, and RTU frames over TCP or UDP), and keeps everything on your computer.

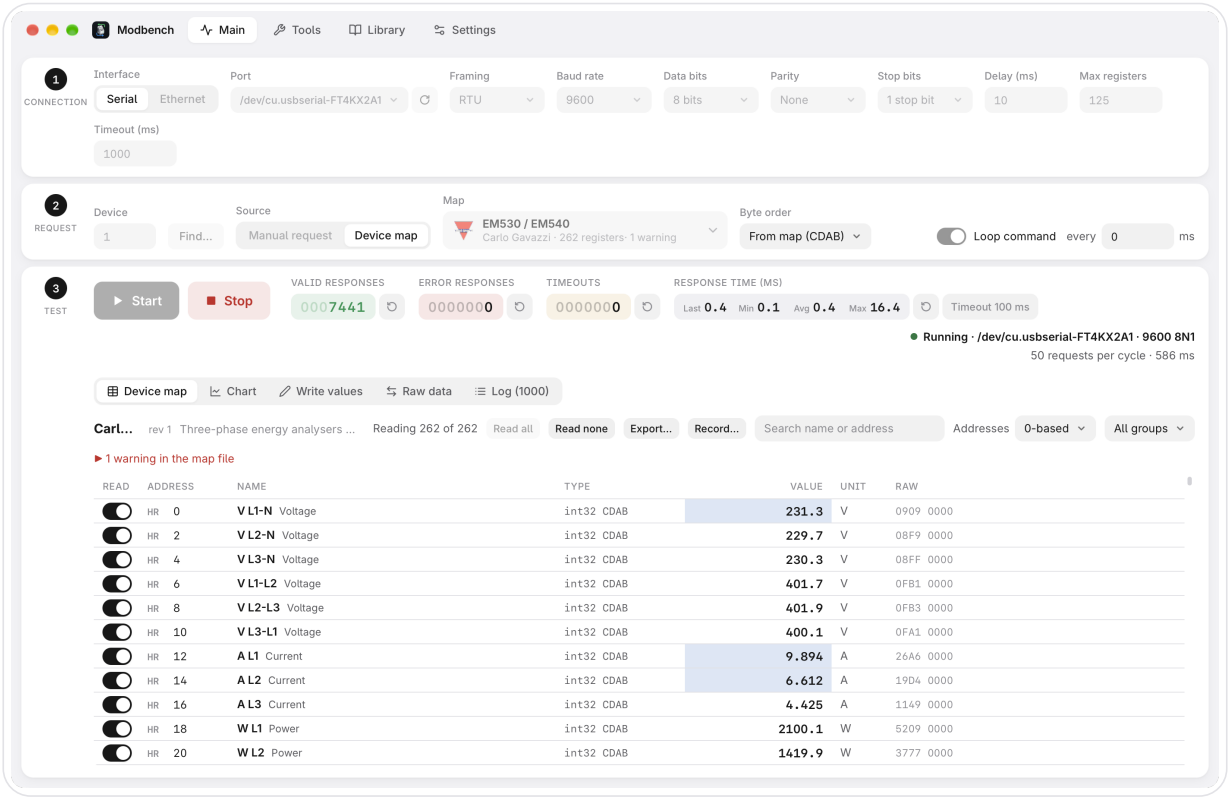


Figure 1. The Main page with a device map open.

1.1 What you can do with it

- **Send any request**: choose a function code, an address and a quantity, press Start, and see the values, once or in a loop.

- **Read a device by name**: open a map (`.mbmap.json`) and every register shows with its name, unit, scaling and state labels. Over 400 maps can be downloaded from the shared library.
- **Check that the map fits**: reference values catch a wrong address, base or byte order before you trust the other values.
- **Write safely**: typed controls, the map's limits, an optional confirmation, and a read-only mode that blocks every write.
- **See what happens**: charts, the raw bytes of each frame decoded field by field, a log of every request, and statistics of failures by cause.
- **Find and identify devices**: scan unit IDs and serial settings, find which addresses answer, ask a device who it is.
- **Simulate devices**: several devices shaped after maps, with values that move on their own, to try maps and other programs.

1.2 A few Modbus words

If you work with Modbus every day, skip this section.

Client and server
The client (Modbench) asks; the server (the device, also called slave) answers. A device never speaks first.

Unit ID
The address of a device on a serial line, or behind a gateway: `1` to `247`. Over TCP to a single device it is often ignored, but must still be sent (usually `1` or `255`).

Areas
Each device has four tables of data:

Area	Size	Read / write
Coils	1 bit	read (FC01), write (FC05, FC15)
Discrete inputs	1 bit	read only (FC02)
Input registers	16 bits	read only (FC04)
Holding registers	16 bits	read (FC03), write (FC06, FC16)

Address
The number of a coil or register in its area, from `0` to `65535` on the wire. Manuals often count from 1, or write the area in front (`40001` is holding register 1, that is 0 on the wire). Modbench shows addresses the way you choose (Section 3.3).

Byte and word order
A register holds two bytes; 32- and 64-bit values take two or four registers. Devices disagree on which byte and which register comes first, so the same bytes can read as very different numbers. Maps say which order a device uses; the Data decoder (Section 10.8) helps find it.

Exception
An answer that says the request failed, with a code: `02` for an address the device does not have, `03` for a value it refuses, and so on (Appendix A.4).

1.3 The window

Four pages sit in the title bar. Click one to show it.

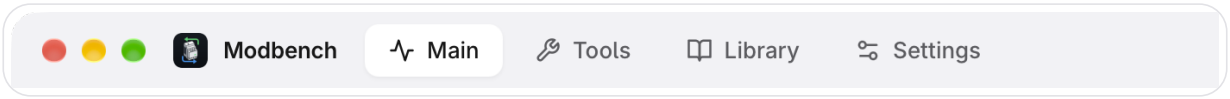


Figure 2. The title bar and its four pages.

Main
Connect to a device and test it, in three steps: *Connection*, *Request* and *Test* (Chapter 3).

Tools
The simulator, and tools to identify devices, find addresses, decode data and frames, play back recordings, see statistics and set alarms (Chapters 9 and 10).

Library
The device maps on this computer, and the shared online library (Chapter 8).

Settings
Theme, text size, safety, what Modbench remembers, updates (Chapter 11).

Right-click menus

Most lists and values have a right-click menu with what can be done there: copy a value or an address, add a register to the chart, write a value, export, remove. Text fields have **Cut**, **Copy**, **Paste** and **Select all**.

The command menu

`Ctrl+K` (`Cmd+K` on a Mac), anywhere in the window, opens a menu to go somewhere quickly: type a page, a tool or a map of the library, and press `Enter`. The arrows move through the results, and `Ctrl+K` or `Esc` closes it. It also has **Start the test** and **Stop the test**, when they apply.

Light and dark

Modbench follows the system theme, or the one chosen in **Settings** › **Appearance**. The screenshots in this manual use the light theme.

CHAPTER 2

Installation and updates

Installers are on the releases page: `github.com/nobrega8/modbench/releases`. Each release lists what changed.

2.1 Windows

Download `Modbench_x.y.z_x64-setup.exe` (recommended) or the `.msi`, and run it. The installer is about 2 MB: Modbench uses the WebView2 that comes with Windows.

2.2 macOS

Download `Modbench_x.y.z_universal.dmg`: one app for Apple Silicon and Intel Macs. Open it and drag Modbench to **Applications**.

> First start
>
> Modbench is not signed with an Apple Developer ID, so macOS asks before the first start. Right-click the app in **Applications**, choose **Open**, then **Open** again. Later starts need nothing.

2.3 Linux

- **AppImage**: download it, make it executable (`chmod +x Modbench*.AppImage`) and run it.
- **Debian, Ubuntu and derivatives**: install the `.deb`, or add the APT repository so that `apt upgrade` brings new versions. It has packages for `amd64` (PCs), `arm64` (Raspberry Pi 3, 4 and 5 with a 64-bit system, and other 64-bit ARM boards) and `armhf` (32-bit Raspberry Pi OS), for Debian 12 and 13, Ubuntu 22.04 and later and Raspberry Pi OS Bookworm or later; APT picks the one for the machine. Raspberry Pi OS Buster and older cannot run Modbench (it lacks WebKitGTK 4.1).
- **Arch Linux and derivatives** (`x86_64`): each release has an Arch package, `modbench-x86_64.pkg.tar.zst`. It needs no repository; the same command, below, installs and updates it.

```
curl -fsSL https://nobrega8.github.io/modbench/key.asc | sudo gpg --dearmor -o /usr/
    share/keyrings/modbench.gpg
echo "deb [signed-by=/usr/share/keyrings/modbench.gpg] https://nobrega8.github.io/
    modbench stable main" | sudo tee /etc/apt/sources.list.d/modbench.list
sudo apt update && sudo apt install modbench
```

```
sudo pacman -U https://github.com/nobrega8/modbench/releases/latest/download/
    modbench-x86_64.pkg.tar.zst
```

> Serial ports on Linux
>
> To open serial ports without root, add your user to the `dialout` group (`sudo usermod -aG dialout $USER`) and log in again.

2.4 Updates

Modbench checks for a new version when it starts (this can be turned off in **Settings › Updates**). When there is one, a banner says what changed:

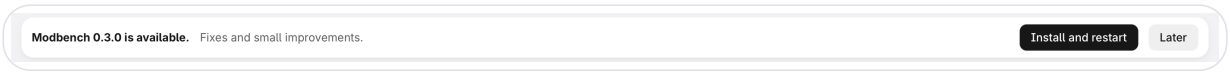


Figure 3. The update banner.

Install and restart downloads the update, checks its signature, installs it and starts Modbench again. Later hides the banner until the next start. **Settings › Updates** can also check by hand.

Installs from the APT repository update with the system (`sudo apt upgrade`), and the Arch package with the `pacman -U` command above, instead.

2.5 Files Modbench opens

The installers register two file types, each with its own icon. A double-click opens them in Modbench, also when it is already running.

File	What it is
`.mbmap.json`	A device map (Chapter 4). Opens on the Main page.
`.mbmap`	A device map in the older version 1 format; Modbench can convert it.
`.mbsim`	A simulator setup (Section 9.6). Opens in the simulator.

> Note
>
> Windows tells file types apart by their last extension only, so `.mbmap.json` files keep the icon of JSON files there. On Linux they get the map icon.

CHAPTER 3

The Main page

The Main page tests a device in three steps, from top to bottom: *Connection* (how to reach it), *Request* (what to ask) and *Test* (send, and see the answers).

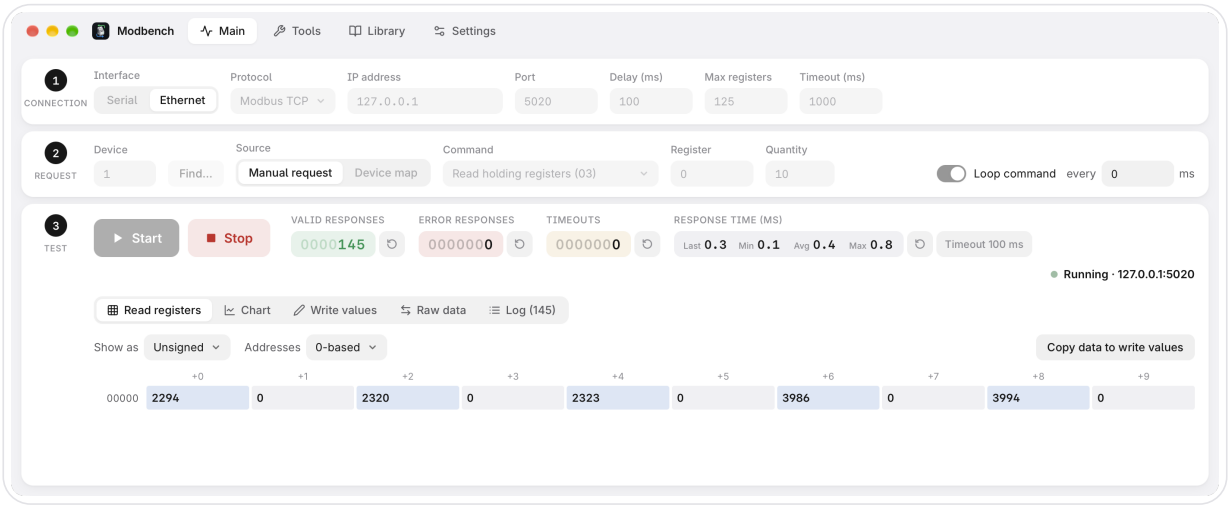


Figure 4. The Main page: Connection, Request and Test.

3.1 Step 1: Connection

Interface

Choose how the device is reached:

Interface	Use it for
Serial, RTU	RS-485 or RS-232 devices: the usual Modbus on a serial line.
Serial, ASCII	Older devices that send frames as text, checked with an LRC.
Modbus TCP	Devices and gateways on Ethernet (port 502).
Modbus UDP	Devices that answer Modbus TCP frames over UDP.
RTU over TCP	Serial-to-Ethernet converters that pass RTU frames as they are.
RTU over UDP	The same, over UDP.

SERIAL

Serial port lists the ports of this computer (↻ looks again after plugging an adapter in). Set **Baud rate**, **Data bits**, **Parity** and **Stop bits** as the device is configured: every device on the line must use the same.

NETWORK

IP address and **Port**. The port is 502 unless the device or the converter uses another.

Send from picks the network interface on a computer with several (Ethernet, Wi-Fi, a VPN): the connection, and the network scan, then send from that interface's address. **Automatic**, the default, leaves the choice to the system's routing. It is saved with the connection. If the address is no longer on the computer, or the interface has no route to the device, the error says so; the step 3 line shows the address used (`192.168.1.10:502 from 192.168.1.5`).

TLS (MODBUS/TCP SECURITY)

With Modbus TCP, **TLS** wraps the connection in TLS 1.2 or later, as the Modbus/TCP Security specification says; turning it on moves the port from 502 to 802, its usual port. Three PEM files go with it:

- **CA certificate**: the device's certificate must be signed by this CA, or the connection fails and says why (an unknown issuer, an expired certificate, another name). Empty trusts any certificate the device shows, which is fine on the bench but not on a network you do not control.
- **Client certificate** and **Client key**: this computer's certificate and private key, for devices that ask for one (the specification's mutual authentication). A device that wants one and gets none closes the connection.

The paths are kept with the connection, and with each device's map; the files are only read when connecting, and keys never appear in the log.

Device

Device (unit ID) is the device's address: `1`–`247` on a serial line or behind a gateway. **Timeout (ms)** is how long to wait for an answer before counting a timeout; **Delay (ms)** is the pause between two requests, which slow devices and some converters need.

> **Not sure of the settings?**
>
> **Find...** scans for unit IDs that answer, or, on a serial line, for the baud rate, parity and stop bits of a device (Section 3.5). The Statistics tool suggests a timeout from the measured response times (Section 10.12).

3.2 Step 2: Request

Source chooses between a *manual request* and a *device map*.

Manual request

Command
The function code: read coils (FC01), discrete inputs (FC02), holding registers (FC03) or input registers (FC04), or one of the writes (FC05, FC06, FC15, FC16).

Start address
The first address, in the notation chosen in Settings (below).

Quantity
How many coils or registers. Modbus allows up to 125 registers or 2000 bits per read.

Device map

With **Source** set to a map, **Map** chooses one of the maps of the library, an open file, or **From a file…**. Modbench then reads every register of the map by name (Chapter 4). The map's connection defaults (unit ID, baud rate, parity, port) fill step 1 when the map is picked; you can still change them.

Byte order

Byte order reads every value of a map with another byte and word order than the map says. It is meant for checking: if the values only make sense with another order, the map, or the device's configuration, is wrong.

Loop

With **Loop command** on, the request repeats until `Stop`. The time next to it is from the start of one cycle to the start of the next; `0` repeats as soon as **Delay** allows. It can be changed while the test runs. Without Loop, **Start** sends the request once.

3.3 Address notation

Manuals write addresses in many ways. The address menu on the `Device map`, `Read registers` and `Log` tabs chooses how Modbench shows them:

Notation	Holding register 100	Notes
0-based	`100`	The address on the wire.
1-based	`101`	As many manuals number registers.
Hex	`0x0064`	As in protocol documents.
Modicon	`400101`	Area digit in front (`0`, `1`, `3`, `4`), 1-based.
IEC 61131-3	`%MW100`	As PLC programs write them.

Address fields also take the Modicon and IEC forms as typed, and switch the command to their area: typing `300010` reads input register 9.

3.4 Step 3: Test

Start sends the request (once, or in a loop); **Stop** ends it. The counters show **Valid responses**, **Error responses** (exceptions) and **Timeouts**; the button next to each counter sets it back to zero.

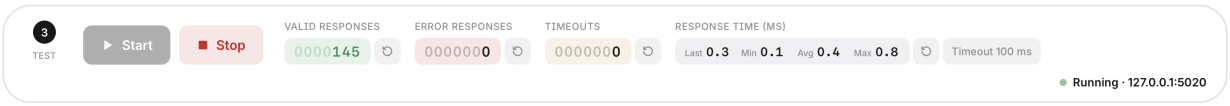


Figure 5. Start, Stop and the counters.

The tabs below show the results:

Device map or **Read registers**
The values: by register name with a map (Chapter 4), as a grid for a manual read (Chapter 5).

Chart
Values over time (Chapter 6).

Write values
Write coils and registers (Section 5.3).

Raw data
The bytes of each frame, decoded (Section 7.1).

Log
Every request with its result and time (Section 7.2).

When the link drops

With **Reconnect when the connection is lost** on (in Settings), a test that loses its link (a cable pulled, a gateway restarted) keeps trying to reconnect, and goes on where it stopped.

3.5 Finding devices

Find... in step 1 opens a scan:

Unit IDs
Asks every unit ID in a range (1–247 by default) and lists the ones that answer. Use a short **Wait (ms)** to go faster on a quiet line.

Serial settings
For a serial device whose settings are unknown: tries the chosen baud rates, parities and stop bits, most common first, until the device answers.

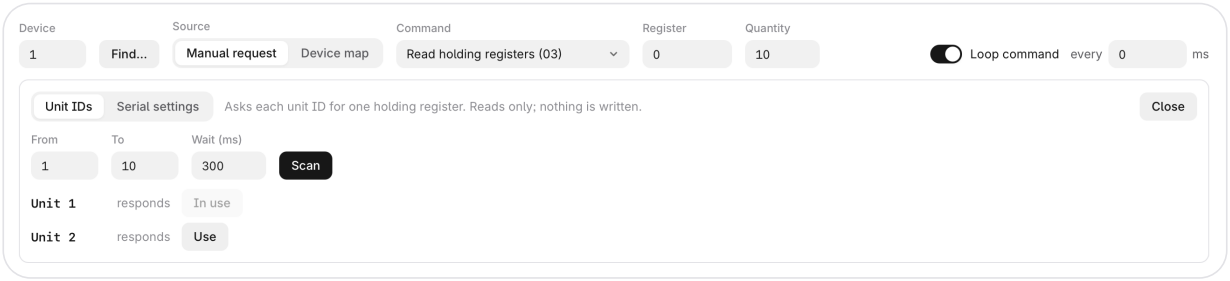


Figure 6. Finding unit IDs and serial settings.

Use next to a unit ID that answered puts it in step 1.

CHAPTER 4

Device maps

A device map describes the registers of a device model: their names, areas, addresses, types, byte and word order, scaling, units and state labels, plus the device's connection defaults. With a map, Modbench reads a device by name, as its manual describes it, instead of as a list of numbers.

Maps are `.mbmap.json` files in an open format. Over 400 of them, for meters, inverters, drives and controllers, are shared in the *mbmap-library* and can be downloaded from the Library page (Chapter 8).

4.1 Opening a map

- In step 2 of the Main page, set **Source** to the map, and pick it in **Map**: the maps of your library are listed there.
- **From a file...** opens any `.mbmap.json` or `.mbmap`.
- Double-click a map file: it opens in Modbench.
- From the Library page: right-click a map and choose **Use**.

Several maps can be open at once; step 2 switches between them. The map's connection defaults (unit ID, port, baud rate, parity) fill step 1 when it is opened.

> Older maps
>
> A `.mbmap` file in the version 1 format opens with a warning, and Modbench offers to convert it: the converted copy is saved next to it as `.mbmap.json`, and the original is kept.

4.2 The Device map tab

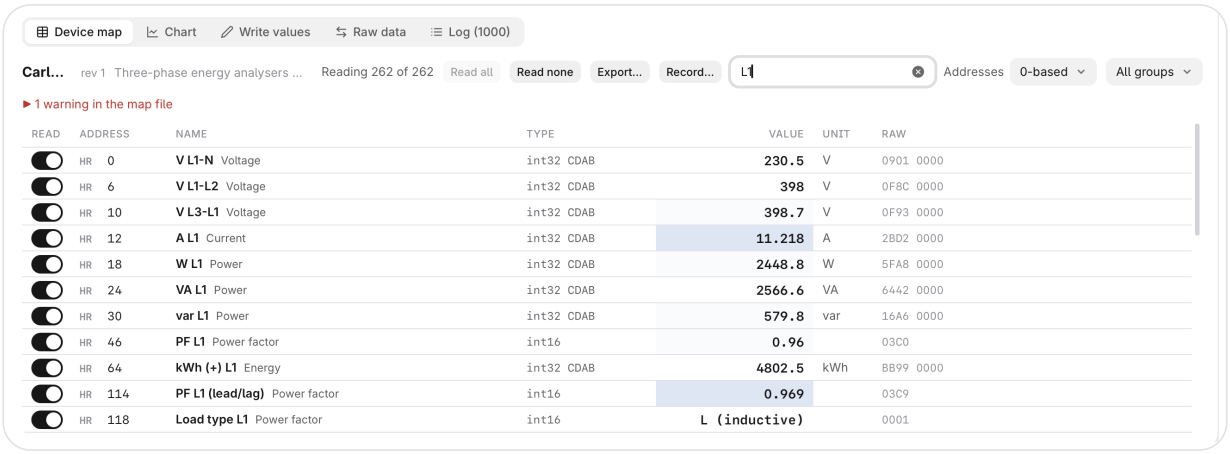


Figure 7. The Device map tab during a test.

Every register of the map is a row: its name, address, value with its unit, and its group. Values update with each cycle of the test.

Choosing what to read
Untick a register to stop reading it; Read all and Read none change them all. Fewer registers make each cycle shorter. Modbench joins neighbouring registers into as few requests as the device allows.

Search
Search name or address filters the list.

State labels
Registers that hold a state (`0` Stopped, `1` Running...) show its label; reserved values the map declares (such as `0xFFFF` for "not implemented") show their meaning instead of a number.

Bits and bit fields
Registers that pack several flags show each bit or field by its own name.

Changed values
A value that changes stays highlighted for a while (**Settings** › **Values**), so changes stand out in a long list.

Right-click
Copy a value, a name or an address, add the register to the chart, or write it.

Registers the map is missing

Add a register the map is missing... adds a register on this computer only: its address, type and name. It is read with the others and marked as added; it is not written into the map file.

4.3 Is it the right map?

A wrong map, base address or byte order gives values that look like numbers but mean nothing. Modbench checks in two ways:

Reference values
A map can give the value a healthy device holds in some registers (a model code, a fixed protocol identifier). Modbench reads them and shows *match*, *mismatch* or *failed* (no answer). A mismatch points to a wrong map or settings.

Models and firmware
When the map says which models and firmware versions it fits, and has registers that identify the device, Modbench compares them with the connected device. *Device recognised* confirms it; otherwise a banner says the device does not match the map. Once the firmware is known, registers that do not exist in that version are left out.

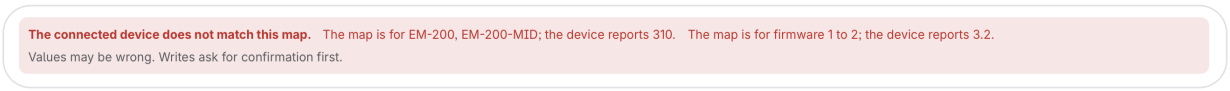


Figure 8. A device that does not match the map.

> **Careful**
> After a mismatch, writes ask for confirmation even when confirmation is off.

4.4 Checking a device against its map

When the connection and the map are set, **Check device** (the tick button in the toolbar of the Device map tab) reads every register of the map once and gives each a result: *passed*, *failed* or *not read*. It also compares the model and the firmware the device reports with what the map declares, like the checks above. The summary counts the three results, or says *No failures*.

Save report (HTML) writes the result to one page, to keep with a commissioning or to send. Check device needs the connection to itself, so stop reading the map first. It only reads, and never writes to the device. The command line does the same with `modbench-cli check` (Chapter 12).

4.5 Map status

Maps from the library carry a status, shown as an icon in the library and in the map picker:

Status	Meaning
Verified	Checked against a real device.
Official	Written from the manufacturer's documentation, not yet checked on a device.
Unofficial	Not from the manufacturer's documentation, for example reverse-engineered.

4.6 Byte order override

Byte order in step 2 reads every value with another byte and word order than the map says: *as the map says*, or one of the four orders. Use it to check a map whose values look wrong; it is remembered per device.

4.7 Computed values

Add a computed value... under the registers adds a value worked out from others: a name, a unit and a formula, for example power from voltage and current, or the difference between two counters:

```
Voltage_L1 * Current_L1 / 1000
[Energy import] - [Energy export]
round(Power / Power_max * 100, 1)
```

A register is named by its name with underscores for spaces and other signs (`Voltage L1` is `Voltage_L1`), or by its exact name in brackets. Formulas take `+ - * / %` and `**` (power), comparisons (`< <= > >= == !=`, which give 1 or 0), `&& ||`, bit operators (`& | ^ << >>`), hexadecimal and binary numbers, and the functions `abs`, `min`, `max`, `avg`, `round`, `floor`, `ceil`, `sqrt` and `pow`. A formula with a mistake or a name the map does not have is refused, saying what is wrong.

A computed value is shown with **calc** instead of an address and its formula as its type. It is worked out every cycle from the values just read and can be charted, recorded and given alarms like a register. When one of its inputs failed, it has no value (a gap) and says which input. Like registers added by hand, computed values are kept for the map file on this computer and never written into it; **Remove** deletes one.

4.8 Exporting and recording

Export...
Saves the values shown, in the order shown, to a CSV file.

Record…

Appends every cycle of the test to a CSV file until recording stops: one row per cycle, one column per register. A read that failed is written `#failed`, and a reserved value its label (`Unimplemented`), so neither is mistaken for a value. The Trend player plays such a file back (Section 10.10) with the same marks as the Chart tab; older recordings, which left failed reads blank, still open.

The CSV separator is set in **Settings** (comma or semicolon, for spreadsheets in languages that use a decimal comma).

4.9 What is remembered per device

For each device (map, connection and unit ID), Modbench remembers which registers were switched off, the byte order override and Loop. They come back when you return to that device; restoring them never writes and never starts a test. **Settings** › **Connection** turns this off, and forgets what was saved.

CHAPTER 5

Reading and writing

5.1 The Read registers tab

With a manual request, the values come back one per row, each with its address (List), or ten per row (Grid), where a value's address is the row's first address plus the column. The choice is kept. Headers stay in view while the values scroll.

A line above the values says whether they can be written: holding registers and coils are `writable`, and their values are outlined like fields; input registers and discrete inputs are `read only`, and so is everything while read-only mode is on. Click a value to select it: it gets a frame, a Address next to its address in the list, and the line shows its address and value, so the address is never in doubt in the grid either. The arrow keys move the selection.



Figure 9. Holding registers read as 32-bit values.

Show as
Shows the registers as unsigned or signed 16-bit numbers, hexadecimal, binary, ASCII text, 32-bit signed or unsigned integers, or 32-bit floats, in the byte order chosen next to it. When the right type is not known, the Data decoder shows them all at once, 64-bit types included (Section 10.8).

Bits
Shows the 16 bits of each register. A bit of a holding register can be switched on its own: Modbench writes the register with only that bit changed.

Addresses
The address menu chooses the notation (Section 3.3).

List, Grid
The layout of the values.

Right-click
Copy a value, an address or all values, add the address to the chart, or **Write here...** and **Write all...**.

5.2 Snapshots

Snapshot… on the Read registers and Device map tabs keeps the values shown now, to answer “what changes when I press this button?” or “what is different between these two units?”.

Take snapshot
Keeps the values shown. A Snapshot column then shows them next to the live values, and every value that differs is marked with ≠ and a bar at the start of its row. The line next to the button counts the differences.

Only differences
Shows only the values that differ.

Save snapshot…, Open snapshot…
Keep a snapshot in a file (`.mbsnap.json`) and open it later, on this or another device.

Compare with a saved snapshot…
Compares the snapshot with a second one from a file instead of the live values; Compare with the live values goes back.

A map snapshot keeps each register by name and address, and a register that was not read on one side is not counted as a difference. A snapshot of a manual read keeps the words from its first address and is compared in the format shown.

5.3 Writing values

The Write values tab writes coils and registers.

Without a map

Choose the function (single or multiple coils, single or multiple registers), the start address and the values, then Write. Copy data to write values fills the list with the values last read, to change one and write them back.

With a map

The map’s writable registers are listed by name, each with a control for its type: a number in engineering units, a list of the map’s labels for state registers, a switch for bits. Modbench applies the inverse of the scaling, checks the type’s range and the map’s limits, and only then writes.

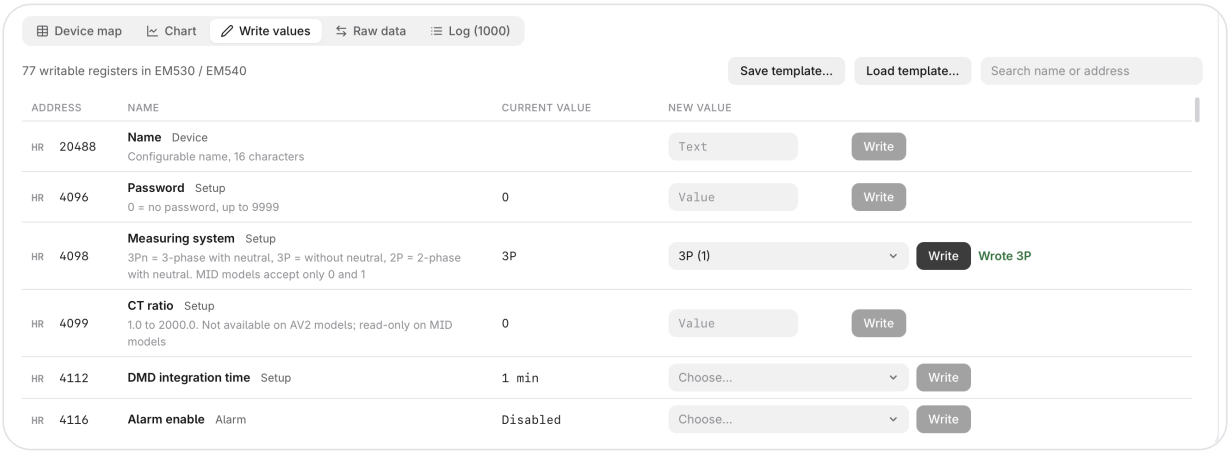


Figure 10. Writable registers of a map; one mode just written.

Limits
A value outside the map's limits is refused before anything is sent, with the limit it broke.

Bit fields
Writing a bit or a field reads the register first and changes only its bits.

Custom values
For a register with labels, a value that is not one of them can still be typed.

Templates

A template copies the settings of one device to another of the same model. **Save template...** saves the values last read from the writable registers; **Load template...** writes the values of a saved template to the device shown.

5.4 Writing safely

Writes change how a real machine behaves. Modbench has three guards, in **Settings › Connection**:

Confirm before writing to a device
Asks before every write: write commands, map values and templates.

Read-only mode
Blocks every write, also on a connection that is already open. Step 3 shows a lock while it is on; click it to go to the setting.

Map mismatch
After a device does not match its map, writes always ask first.

> **Careful**
> A write is sent as soon as it is confirmed. Check the unit ID: on a serial line, unit ID `0` is a broadcast that every device takes.

CHAPTER 6

Charts

The `Chart` tab plots values over time while a test runs: map registers by name, or addresses of a manual read.

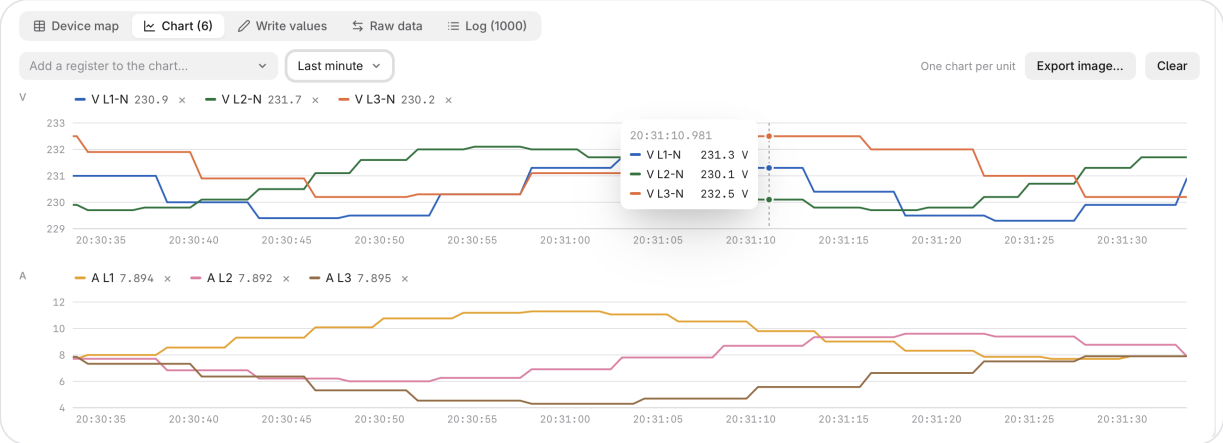


Figure 11. Voltages and currents of a meter, one chart per unit.

6.1 Adding lines

- With a map: **Add a register to the chart** lists the map's numeric registers.
- With a manual read: add an address from the list, or right-click a cell on the `Read registers` tab and choose **Add to chart**.

Up to eight lines can be shown, each in its own colour. Lines with the same unit share a chart, so values of different sizes (volts and amperes) do not flatten each other. The tab's name counts the lines: `Chart (3)`.

Values are recorded from the moment a line is added, while the test runs. To remove a line, click `×` next to its name in the legend.

6.2 Reading the chart

Time span
Time span shows the last minute, 5 or 15 minutes, the last hour, or everything since the lines were added.

Legend
Each line’s name and its newest value.

Hover
Move the pointer over a chart to see every line’s value at that moment.

Spikes
When there are more samples than pixels, each pixel keeps its lowest and highest value, so short spikes stay visible.

Gaps
A failed read leaves a gap in the line instead of a false value, and a mark along the bottom of the chart says why: `×` (in red) where the read failed, `|` where the device gave no value (a reserved value such as `Unimplemented`). Pointing at a gap shows the reason and the last good value, marked `(held)`. When map registers start failing or read again, the Log tab says which ones and why.

`Clear` empties the charts and keeps the lines.

6.3 Markers

Click a chart to place marker `A`, and again to place `B`; after that, a click moves the nearer marker, and a marker can be dragged. Both markers are times, shared by every chart of the tab, and stay where they are while new values come in. Under the charts, a table gives for each line the value at `A` and at `B`, the difference `B − A`, and the minimum, maximum and average of the good samples between them, with the time between the markers (`Δt`). `Clear markers` removes them. The Trend player has the same markers.

6.4 Saving a chart as an image

`Export image…` copies or saves the charts shown, for a report or a bug report:

Choice	What you get
Copy image	The same picture as PNG, on the clipboard, to paste into a ticket, an e-mail or a document. `Copied.` says it worked.
Copy image, light theme	As Copy image, in the light theme.
PNG…	A picture at twice the screen size, sharp in documents.
SVG…	A vector drawing that scales to any size and opens in other programs (colours written out, no styles needed).
PNG, light theme…	As PNG, in the light theme even when Modbench is dark.
SVG, light theme…	As SVG, in the light theme.

The image holds every chart shown, each with its unit and legend, under a line with the

date and the time span. The hover marks are left out. The image is taken when you choose, so it does not move on while the save dialog is open.

The Trend player (Section 10.10) has the same button.

CHAPTER 7

Raw data and the log

When a device answers strangely, the frames tell why. These two tabs show what was really sent and received.

7.1 The Raw data tab

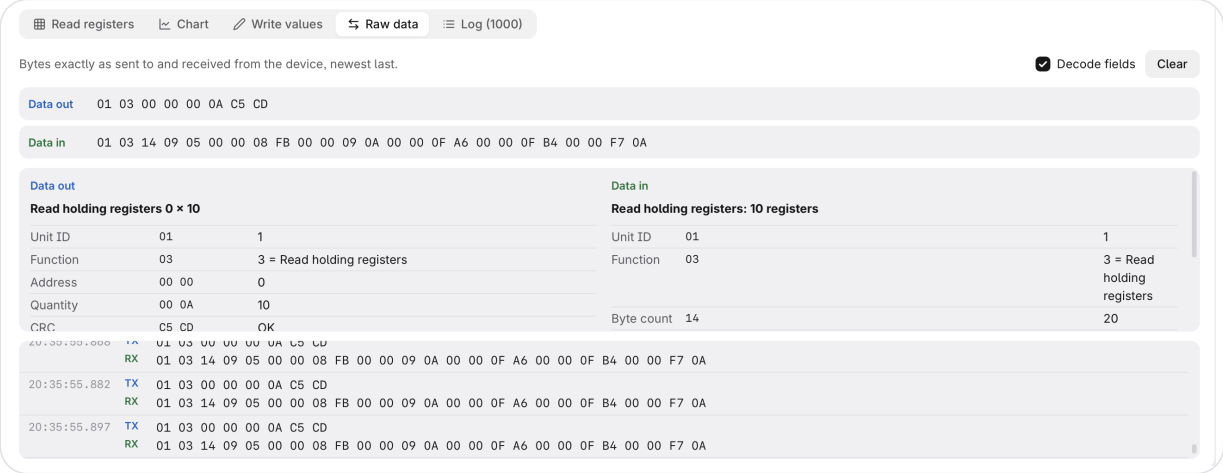


Figure 12. A request and its answer on the Raw data tab.

The bytes exactly as sent to and received from the device, newest last: `Data out` in blue, `Data in` in green, as in the logo. Each frame is decoded field by field (unit ID, function code, address, quantity, byte count, data), and its check is verified: the CRC on RTU, the LRC on ASCII, the header on TCP and UDP. A frame that fails its check is marked.

When no answer came, the tab says so (*No response.*). Clear empties it.

> **Tip**
>
> To read a frame from another program or a bus analyser, paste it into the Frame decoder (Section 10.9): it decodes it the same way.

7.2 The Log tab

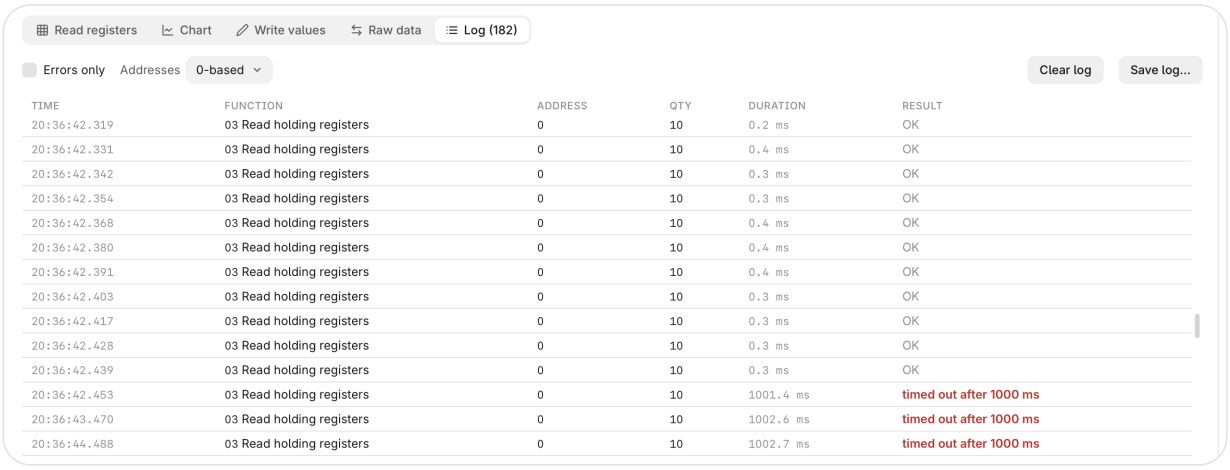


Figure 13. The Log tab, with three timeouts in red.

Every request: **Time**, **Function**, **Address**, **Result** (OK, the exception, or a timeout) and **Duration**. The tab's name counts them: `Log (120)`.

Save log...
Saves the log to a CSV file.

Clear log
Empties it.

Right-click
Copy line, **Copy error**, **Copy all**, and **Show errors only** to keep just the failed requests in view.

Size
Settings › **Connection** › **Requests kept in the log** sets how many requests are kept; older ones are dropped first.

For failures by cause over a long test, see Statistics (Section 10.12).

CHAPTER 8

The device library

The Library page keeps your device maps in one folder, and brings maps from the shared online library.

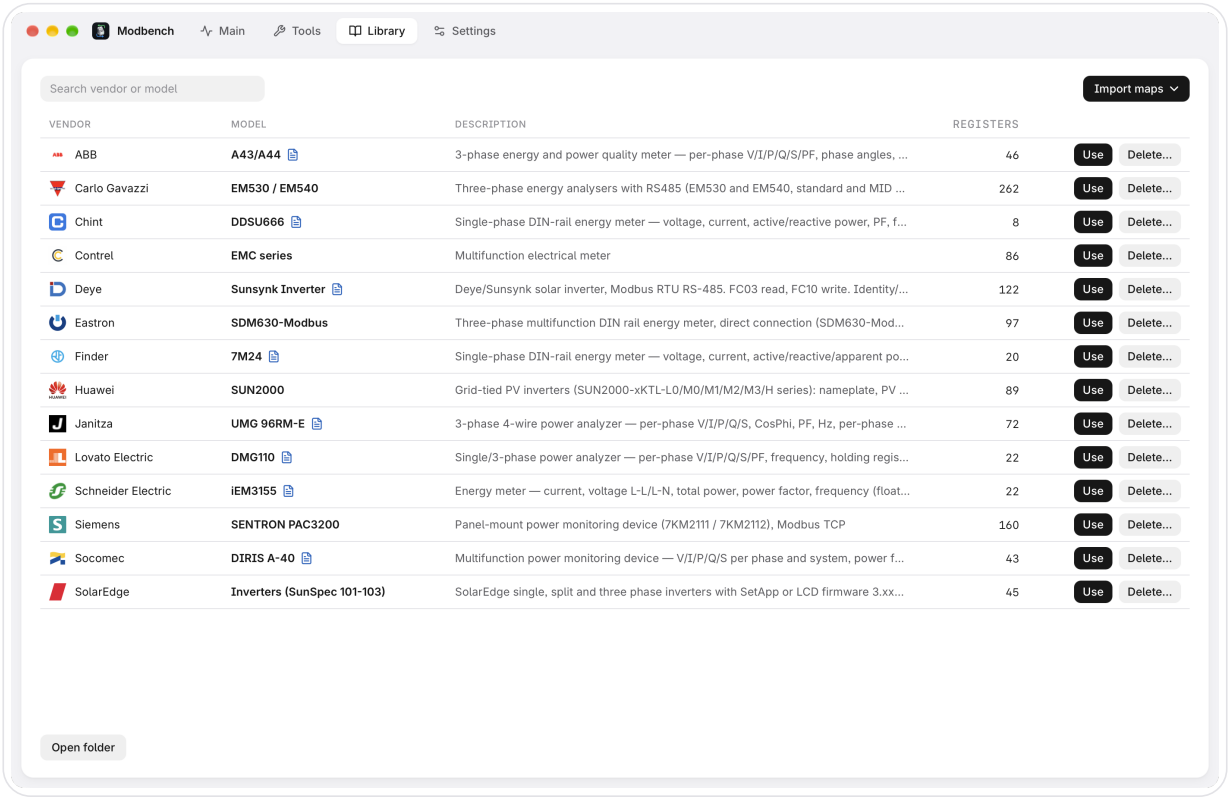


Figure 14. The Library page.

8.1 Your library

The list shows each map's vendor (with its icon), model, description, number of registers and status (Section 4.5). **Search vendor or model** filters it.

Right-click a map:

Use
Opens it on the Main page.

Explore
Shows the registers of the map, with a search, and its properties, translation tables, no-data values and sources, without using it.

Edit…
Opens the map in the map builder (next section).

Open in a text editor…
Opens the file in your text editor, after asking.

Duplicate
Makes a copy of the map in the library.

Simulate this device
Opens the simulator with a device shaped after this map (Chapter 9).

Export for other software…
Writes the map as the Modbus configuration of Home Assistant, sensors for ESPHome, an input of Telegraf, a Node-RED flow (with node-red-contrib-modbus) or a CSV tag list for PLC and SCADA tools. The device's address comes from the connection of step 1, or is left as `<host>` to fill in. Registers a format cannot express are listed at the top of the file.

Copy name, Copy file path
For notes and other programs.

Open the library folder
Shows the folder in your file manager.

Delete…
Removes the map from the library, after asking.

Maps with errors are listed with the problem, so they can be fixed in a text editor.

8.2 Adding maps

From a file…
Copies `.mbmap.json` or `.mbmap` files into the library. Version 1 files are converted.

From the online library…
Opens the shared library (below).

The library is a folder (**Settings** › **Device library** › **Library folder**): files copied into it by hand appear too.

8.3 The map builder

Map builder (in the Library's toolbar) opens a window of its own, next to the main one, to write a map without a text editor. **Edit…** in a map's right-click menu opens that map in it. **New map** starts an empty one, **Open a map…** loads a file.

The window has the map's header and a table of its registers. It checks the map with the same rules as a file while you type, and says so at the top: *No problems found*, or the number of errors and warnings, with the cause beside each one. *Unsaved changes* shows when the map differs from what is on disk.

Device
Vendor, model and the other fields that name the device, the source it was made from (manual, version, URL) and the connection defaults that are filled in when the map is used.

Optional parameters
The firmware the map is for (**Firmware from** and **Firmware up to**), the date it was revised, the most registers or bits one read may ask for, the models it covers and notes, one per line. Sources lists the documents the map was made from. Named state labels, the no-data values by type and any other keys of the file (the ones that begin with `x_`) are here too.

Address base
How the manual numbers its addresses, from 0 or from 1. Addresses in the table are typed as the manual writes them.

Registers
One row for each register. Add a register adds a row; each row can be moved up or down, duplicated or deleted, and Edit register… opens all of its fields. State labels are written as pairs, such as `0=Off, 1=On`.

Changes from the original
Lists what was added, removed or changed since the map was opened.

Save as… writes a version 2 `.mbmap.json`. A map that came from the library is saved back to its file with Save, or copied elsewhere with Save as…; errors must be fixed before it goes into the library, and Modbench asks before it replaces a map the library already has for the same vendor and model. Open in a text editor… opens the file instead.

Importing a register table

Manuals list registers in tables. Import a table…, in the map builder, turns one into the registers of a map. Paste the table copied from a spreadsheet or a manual (`Ctrl+V`), or open a CSV file.

Modbench reads the header to guess what each column holds (name, group, area, address, type, length, scale, unit, access or description). Each guess can be changed, a column can be left out, and **The first row is a header** is ticked when the first row looks like one. Types, scales and access are read the way manuals write them (`x0.1`, `1/10`, `R/W`), and so are addresses such as `40001`. Rows that do not say which area they belong to (coils, holding registers…) use **Area of rows that name none**. Addresses are read the way the manual numbers them, that is, with the map's address base.

A cell that could not be read keeps the default value and is marked in red, and the message after the import names the rows to check. Replace the registers swaps the map's registers for the table; Add to the map keeps them and appends the rows.

8.4 The online library

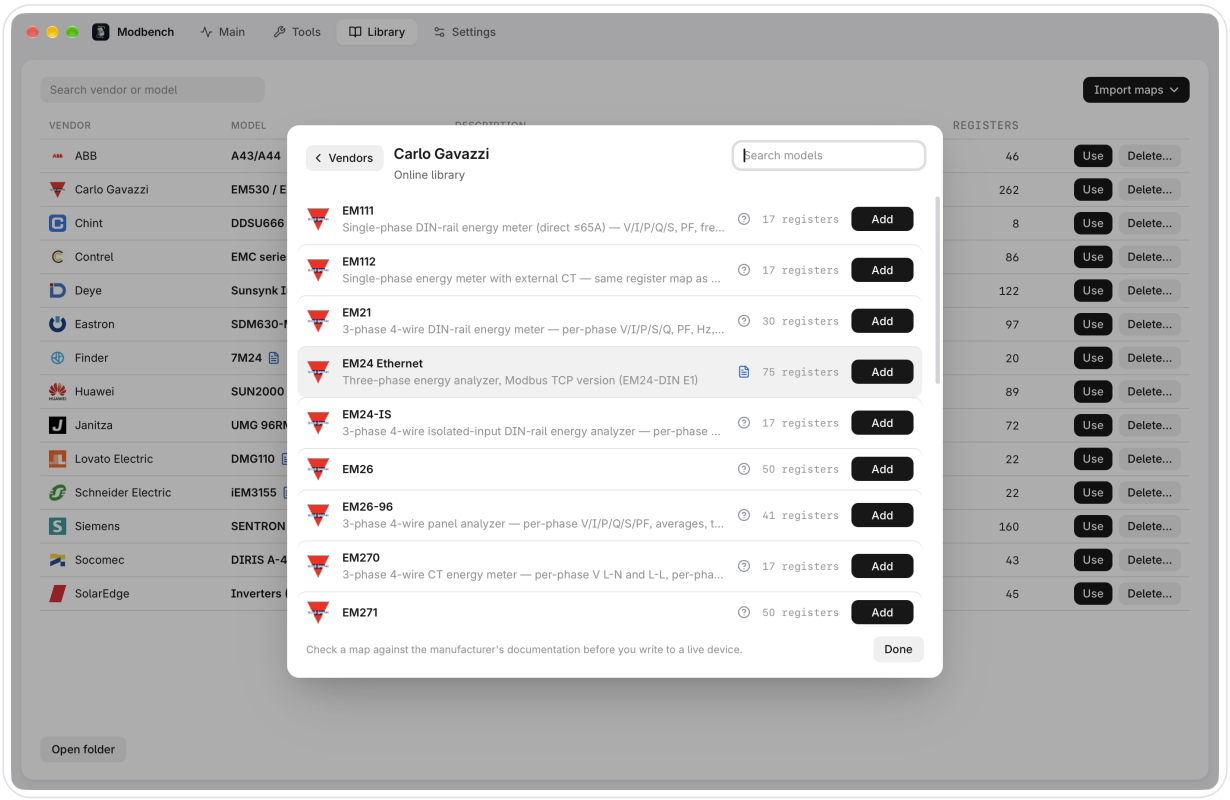


Figure 15. Choosing a map in the online library.

The online library lists the maps shared on GitHub (`github.com/nobrega8/mbmap-library`): over 400 maps of meters, inverters, drives and controllers.

1. Choose a vendor; Back to all vendors (or `Esc`) goes back.
2. Add next to a model downloads its map into your library.
3. Done closes the online library.

A map already in your library can be downloaded again, to replace your copy with the newest one.

> **Share your maps**
>
> A map that works with a real device helps the next person with the same device. The library's repository explains how to contribute one, and the format specification is linked from Settings › Device library.

8.5 Finding the map for a device

Find under Find a map for this device checks the maps of the library, and the maps open on the Main page, against the device of step 1 (it connects for the check if needed, and only

reads). Many maps declare identity registers: the device's model, its firmware version, or registers with a known reference value. Each distinct identity register is read once, however many maps declare it, and each map is compared with what the device answered:

- a model register must name one of the map's models;
- every reference value must match;
- a firmware outside the map's range is shown as a warning, not ruled out.

The maps that fit are listed best first, with what matched (`model EM540`, `2 references`, `firmware 2.1`); Open opens one. Maps without identity registers cannot be checked and are not listed.

The simulator, shaped after a map, answers its model register with the map's first model, so a map can be tried out with this search too.

CHAPTER 9

The simulator

The simulator runs Modbus devices inside Modbench. Use it to try a map, an alarm or another program (a SCADA, a PLC, a script) without the hardware: each device answers like the real one, with the values of its map, and can make values move on its own.

Open it from **Tools** › **Simulator**.

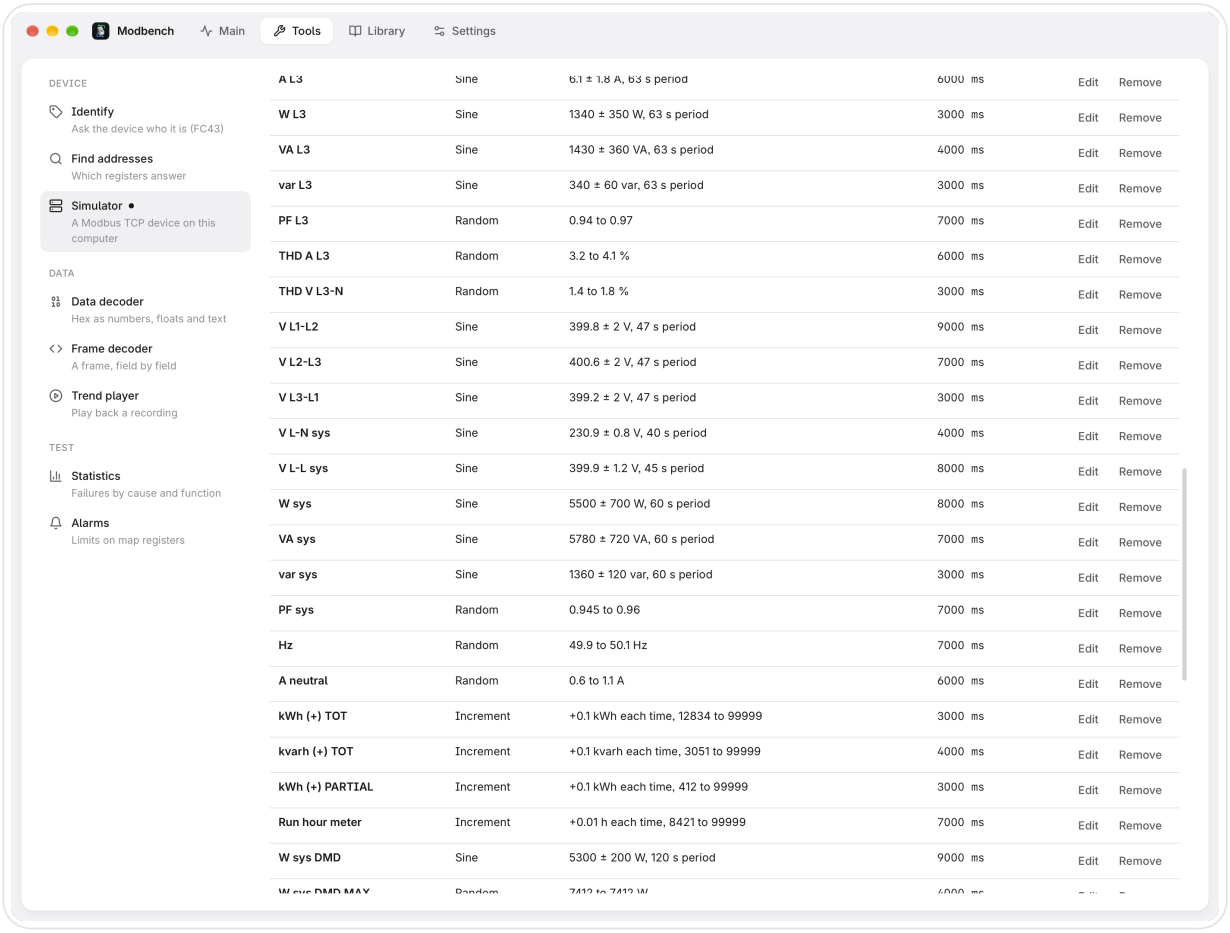


Figure 16. The simulator with two devices on a serial port.

9.1 Where it listens

Interface
Modbus TCP or UDP, RTU over TCP or UDP, or a serial port with RTU or ASCII. Over the network, set the **Port** (5020 by default, so it does not need administrator rights); **Other**

computers also accepts clients from the network, not only from this computer. On a serial port, set the line settings as the clients use them.

Start and **Stop**
Start listens; the status shows where (*Listening on 127.0.0.1:5020*).

Connect Main to it
Fills step 1 of the Main page with the simulator's address and the shown device's unit ID, and goes to Main.

> Serial
>
> Clients connect to the other end of the line. To try it on one computer, use a virtual null-modem pair (com0com on Windows, `socat` on Linux and macOS) and open the other port on the Main page.

9.2 Devices

The simulator starts with one device. Several devices can share one address, each on its own unit ID, like devices behind a gateway or on one RS-485 line; other unit IDs get no answer.

Add device
Adds a device on the lowest free unit ID. The tabs above switch between devices.

Unit ID
The unit ID the device answers. Empty answers every unit ID that no other device has. Unit ID `0` is a broadcast: every device takes the write and none answers.

Map
Shapes the device after a map: one open on the Main page, one from the library, or **Open a map file...**. *Empty device* gives a memory where every address exists and holds zero.

Only the map's addresses
Addresses the map does not have answer exception 2, like the real device. Off, every address answers.

Reset values
Puts back the map's reference values.

Remove device
Removes the device shown; its unit ID gets no answer.

9.3 Values

The view switch shows the device's memory either as **Registers**, the map's registers by name with their values in engineering units, or as a raw **Grid** of addresses, in any area. Click a value to change it: clients read the new value from the next request.

Writes from clients show up immediately, and the recent requests are listed at the bottom, reads and writes apart, so a stream of reads does not push the writes out of sight.

9.4 Generators

Generators make values change on their own, every so many milliseconds (at least 50). Add generator adds one; it drives a map register, in its own units, or a raw address.

Kind	What it does
Increment	Adds a step each time, from a minimum to a maximum, then starts again: counters, energy.
Sine	Swings around an offset with an amplitude and a period: voltage, temperature.
Random	A new value between a minimum and a maximum each time: noise.
Copy	Takes the value of another address: a status input that follows a command coil.

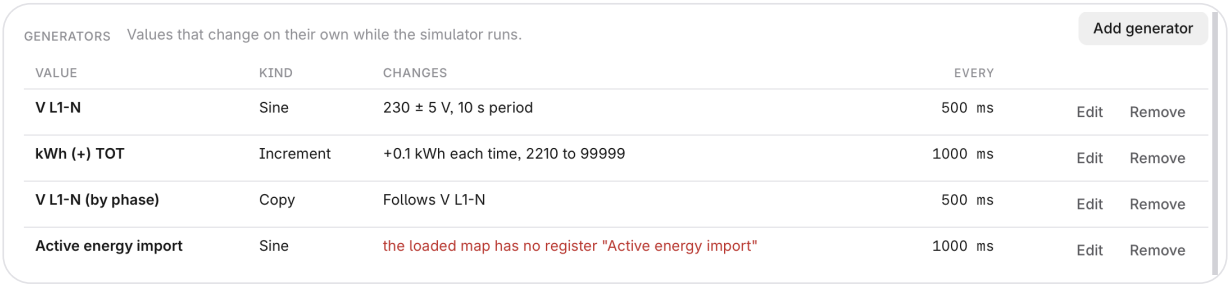


Figure 17. Generators on a meter's registers.

A generator that cannot run (its register is not in the map, or its address does not exist) says why. Generators keep running across Stop and Start.

9.5 Its own window

Pop out moves the simulator into a window of its own, to watch it next to the Main page. Back to the main window brings it back to the Tools page.

9.6 Saving a setup

A test bench with several devices, maps and generators takes a while to set up. Save setup... keeps it in a `.mbsim` file:

- the interface and its settings;
- every device: unit ID, map, *Only the map's addresses*, and generators.

Open setup..., or a double-click on the file, puts it back. Each map is read again from its file when that is still there, so updates to the library are picked up; otherwise the copy kept in the setup is used. While the simulator runs, opening a setup replaces the devices but keeps the interface.

> Note
> Values changed by hand are not saved: devices start from their maps' reference values.

9.7 The demo

The demo tries Modbench without hardware or a map: **Settings › Connection › Demo › Start demo**. With Offer the demo at the first start turned on (it is off by default), the Main page also offers it until a connection is used (Start demo, Not now, Don't ask again).

It loads a made-up meter, `Demo meter (simulated)`, into the simulator, with values that move, starts the simulator on a free port of this computer (5020 or the next free one), and reads it on the Main page with its map, with one value on the chart. A banner says that the device is simulated and points to the Device map, Chart and Raw data tabs. Leave demo stops reading and the simulator and puts back the connection, the request and the simulator's own device as they were. Nothing the demo changes is remembered. The demo does not start while the simulator runs with a setup of your own.

CHAPTER 10

Tools

The Tools page groups the tools by what they work on: the *device*, the *data*, and the *test*. A dot next to a tool means it has something going on (the simulator running, a scan in progress).

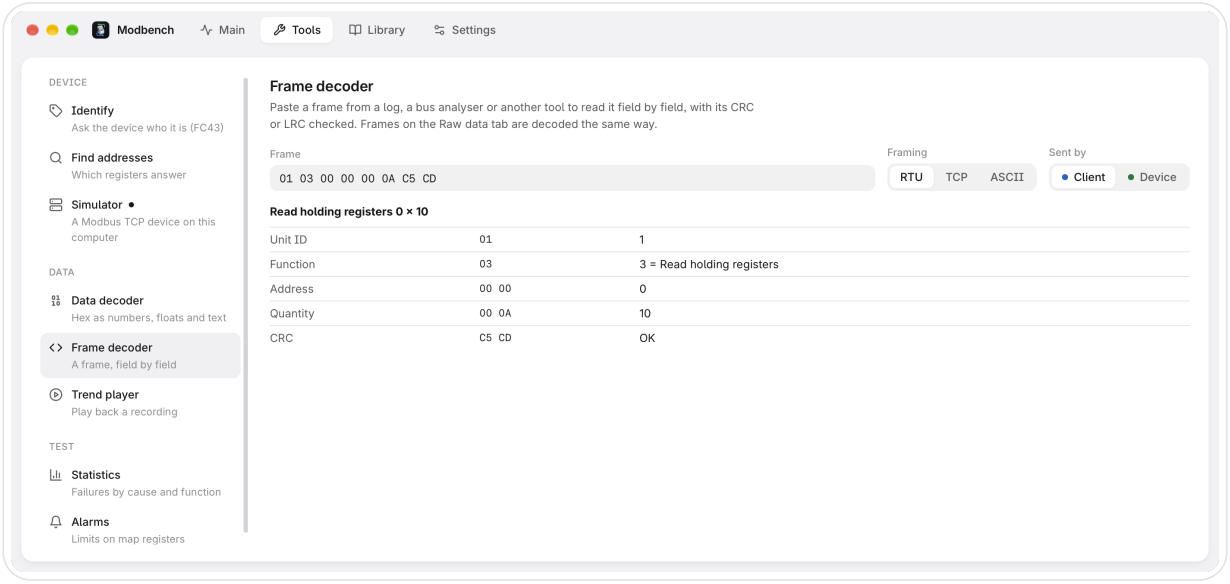


Figure 18. The Tools page.

Tools that talk to a device use the connection of step 1 on the Main page, and only read.

10.1 Identify

Asks the device for its vendor, product code and revision with *Read Device Identification* (FC43/14). Choose which **Objects** to ask for (basic, regular or extended) and Read identification. Each object the device returns is listed with its value.

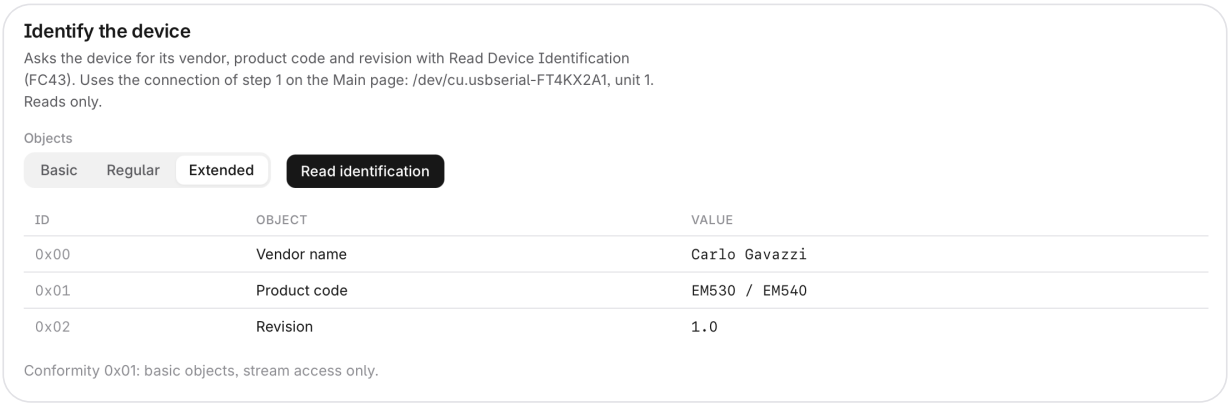


Figure 19. A device that answered FC43.

Not every device supports FC43: many answer exception 1 (illegal function).

Check function codes sends one harmless request for each read function (FC 1–4, 7, 8, 11, 17, 20, 24 and 43) and shows how the device answered: *Supported*; *Not supported* when it answered "illegal function"; *Supported, refused this request* when it answered another exception, which means it has the function but not, for example, address 0; or *No answer*. Nothing is written. FC 7, 8, 11 and 17 exist on serial lines only, so over TCP they often answer "illegal function".

The functions are tried at the **Address** given (0 at first). With Also try write functions, Modbench also tries FC 5, 6, 15, 16 and 23 by reading coil and holding register **Address** and writing the same values back, and FC 22 with a mask that keeps every bit. It always asks first, and the option is off in read-only mode. A write whose value could not be read is *Not tried*. The values do not change, but a device can still act on a write: only do this on a device you may change.

Find largest read finds how many registers or bits the device returns in one read from **Address**, by halving, up to 125 registers or 2000 bits, and says why one more failed. "Illegal data value" means that is the device's limit: set it as **Max registers** on the Main page. "Illegal data address" means the area has a gap there.

10.2 Find addresses

Finds which addresses answer on a device without a manual. It reads blocks of addresses, and splits the blocks that fail until it knows which addresses exist.

Area, From, To
What to search.

Block
Addresses read at once before splitting. Larger blocks are faster on devices with long runs of registers.

Wait (ms)
How long to wait for each answer.

The result lists the ranges that answered. Read next to a range sets up a manual request for it on the Main page.

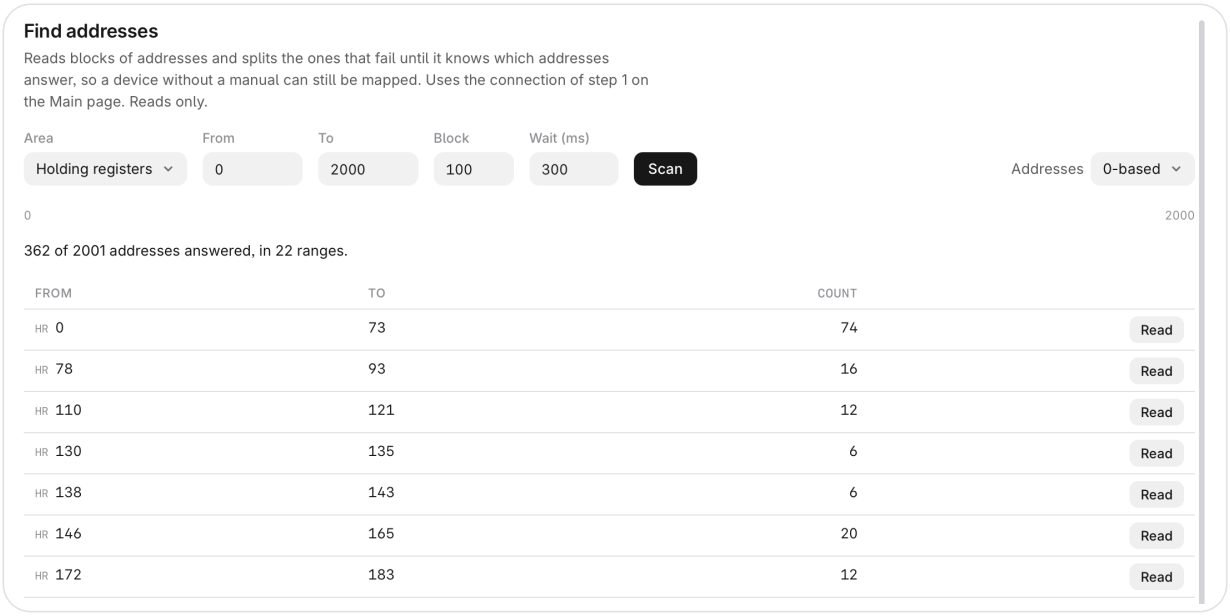


Figure 20. Address ranges found on an energy meter.

10.3 Network scan

Finds Modbus TCP devices on a subnet. It needs no connection on the Main page and leaves an open one alone. Each address gets a TCP connection to the port; when it opens, the scan asks for one holding register. Any answer, an exception included, shows a Modbus device. Nothing is written.

Addresses
A subnet (`192.168.1.0/24`), a range (`192.168.1.10-50` or `192.168.1.10-192.168.2.20`), or addresses separated by commas. Up to 65536 addresses (a /16).

Port, Unit ID
Where to ask, and which unit ID the request carries.

Mode
Fast waits a short time (150 ms by default) and does not retry: a quick first pass to find the devices that are online. Thorough waits longer (1000 ms) and asks once more after a timeout, for slow devices, gateways and busy networks. The line under the settings says which wait and retries the scan will use.

Wait (ms)
How long to wait for the connection, and then for the answer, in the chosen mode; each mode keeps its own.

Send from
The network interface, shared with the connection of step 1.

Identify (FC43)
Also reads the vendor, product code and revision of the devices found.

Results appear as they are found. Modbus marks a device that answered; Port open an address that accepted the connection but sent no Modbus answer in time. Open sets up the connection of step 1 for a device. Stop ends the scan and keeps what it found. An address that does not answer within the wait is left out; that does not prove there is no device there, so try a longer wait on a slow network.

10.4 Exporting scan results

The unit ID and serial settings scans (Find... in step 2), Find addresses and Network scan have Export CSV... once there are results. The file starts with a few lines that say which scan ran, when, and with which settings and connection; a blank line; then one row per result. A stopped scan says where it stopped. The unit ID scan lists every unit ID, with `answered`, `no answer` or `not asked`; the other scans list what answered and count the rest. The network scan can be exported while it runs, without stopping it. Files are UTF-8, with the CSV separator of the settings.

10.5 Gateway

The gateway passes requests from clients on one link to the device of step 1 on another, and the answers back, like a serial-to-Ethernet gateway on the bench: Modbus TCP clients can reach an RTU device on a serial port, or the other way round. Choose where clients connect (Interface: Modbus TCP or UDP, RTU over TCP or UDP on a port, or RTU or ASCII on a serial port) and press Start. Other computers lets clients on the network connect; otherwise only this computer can.

Requests go to the device one at a time. A device that does not answer comes back to the client as exception `0x0B` (gateway target device failed to respond); a link that cannot be opened, as `0x0A` (gateway path unavailable). The link opens with the first request and again whenever it is lost. Send to unit sends every request to one unit ID instead of the client's, and Refuse writes answers writes with exception `0x01` instead of passing them on.

Each request is listed with the client, the unit ID (and where it went), the function, the address, the result and how long the device took. Right-click a row to copy it or every row.

10.6 Bus monitor

The bus monitor listens to a serial line where a master (a PLC, a SCADA, a gateway) already talks to its devices, and shows what they say. It never sends a byte, so it can sit on a working RS-485 line with a second adapter. Choose the serial port, RTU or ASCII, and the line's baud rate, parity and stop bits, and press Start.

Each request is listed with its answer: the unit ID, the function, the address and count, the

values read or written, and the time the device took. A request nobody answers within a second shows `No answer`; an exception shows its code; a frame whose CRC or LRC does not hold shows `Bad CRC/LRC`. RTU frames are told apart by the silence between them, as the protocol says; when the adapter hands over two at once, they are split where a CRC holds.

Filter by unit ID, by function, or to problems only. `Export CSV...` saves the requests the filter keeps, with their bytes; hover a row to see them, or right-click it to copy it.

10.7 Send request

Sends any request to the device of step 1, also the functions step 2 does not offer, and requests built by hand for unusual tests.

Function
Pick a function code (01–24, 43) and fill in its fields: for Mask write register (22), the address and the AND and OR masks; for Read/write multiple registers (23), the read address and count, the write address and the values; for Diagnostics (08), the sub-function and data. `Other function code...` takes any code and its data in hex. The line below shows the PDU that will be sent. `Broadcast (unit 0)` sends to every device: they act, but none answers.

Raw bytes
Bytes exactly as typed, without framing: no MBAP header and no CRC are added. They go out on a link of their own (the connection is closed first, so a serial port is free), and what comes back within **Wait (ms)** is shown.

The frames sent and received are shown, decoded field by field, and appear on the Raw data tab and in the log. `Add to list` keeps a request; `Send all` sends the list once, `Loop` again and again.

A request that may write (any function other than the reads, the diagnostics echo, broadcasts and raw bytes) is not sent while read-only mode is on, and asks first when `Confirm before writing` is on.

10.8 Data decoder

Type or paste registers (in hex) to see them as every type at once, in all four byte orders: signed and unsigned 16-, 32- and 64-bit integers, floats, and text. `Use last read` takes the registers last read on the Main page.

This is the quickest way to find a value's type and byte order before writing a map: look for the column where the number makes sense (a voltage near 230, a frequency near 50).

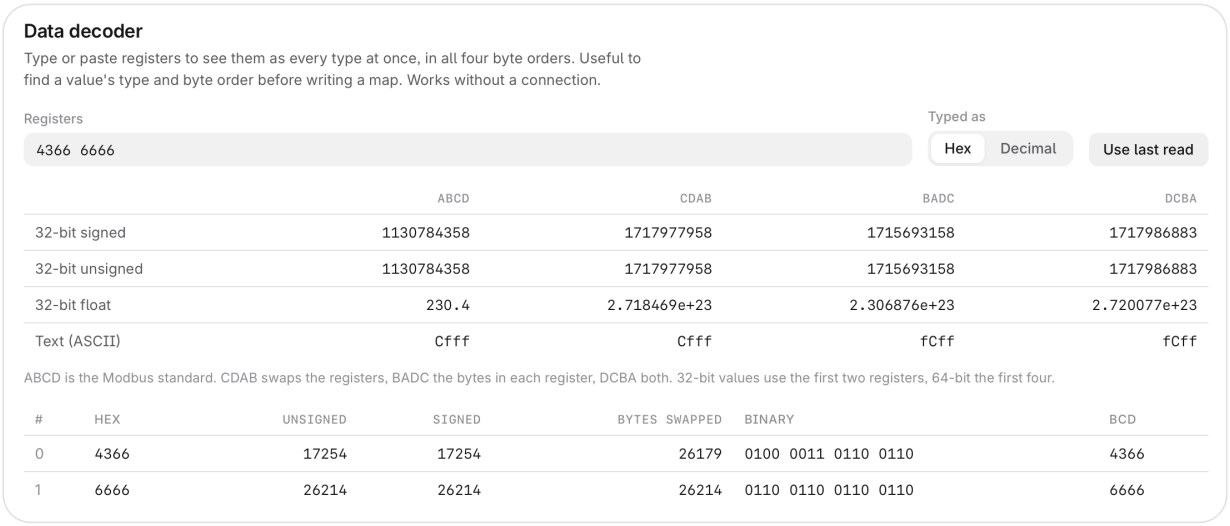


Figure 21. Two registers decoded in every type and order: only 32-bit float in ABCD order reads as a voltage, 230.4.

It works without a connection.

10.9 Frame decoder

Paste a frame from a log, a bus analyser or another tool (for example `01 03 00 00 00 0A C5 CD`) to read it field by field, with its CRC or LRC checked. Choose the **Framing** (RTU, ASCII, TCP) and who sent it (**Client** or **Device**), since a request and an answer with the same function code have different fields.

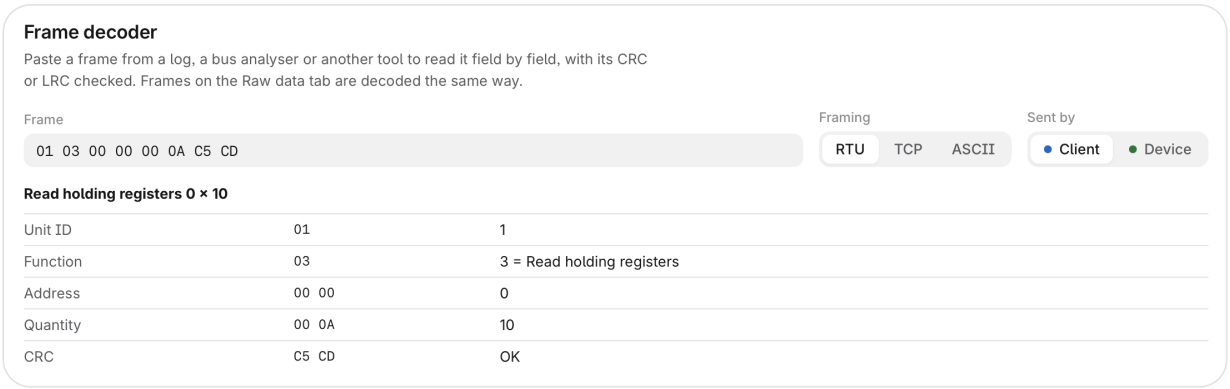


Figure 22. An RTU request decoded.

10.10 Trend player

Plays back a CSV file made with **Record...** on the Device map tab, so a test can be looked at again later.

1. **Open recording...** and choose the file.
2. Tick the columns to plot (up to eight; text columns cannot be plotted).
3. **Play** at 1× to 60× speed, or drag the slider to any moment.

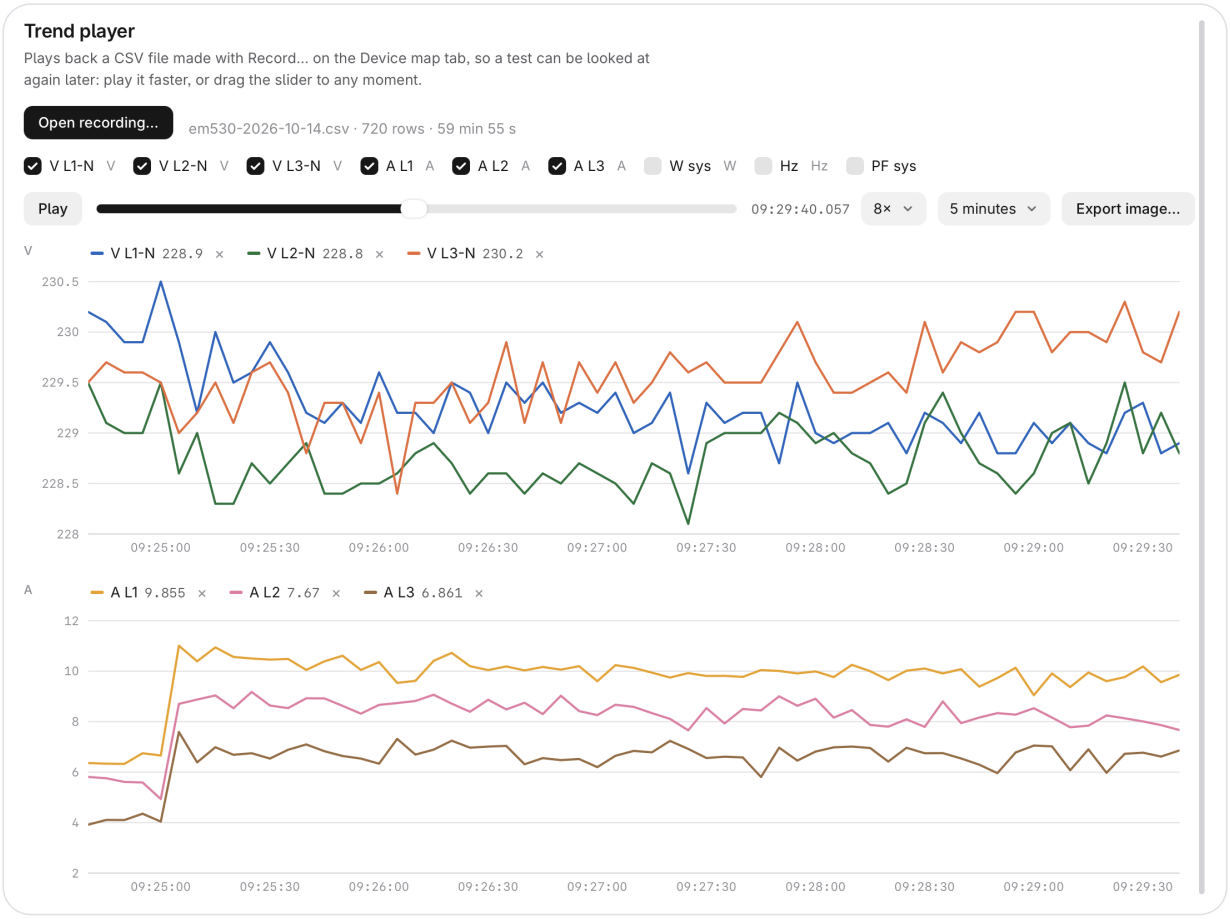


Figure 23. A recording played back.

Time span works as on the Chart tab, and **Export image...** saves the charts (Section 6.4).

10.11 Test sequence

A test sequence checks that a device reacts as it should, which a reading of its registers cannot tell: a setpoint written shows up in a status, a command starts something, a value settles in a range. Open the device's map on the Main page first: the steps name its registers.

Each step is one of:

Write
a value in the register's own units (1 or 0 for on/off).
Wait
some milliseconds.

Expect
the register to equal a value (within a tolerance), or to be in a range, reading it again until it does or the time given runs out (0 reads once).

Read
the register, to record its value in the report.

Run runs every step in order against the device of step 1, and each step shows its result as it ends: Pass or Fail for an expectation, with the value seen; Done for the others; Error when the step could not run (a register not in the map, no answer). A failed step does not stop the ones after it. A sequence that writes asks before it runs, and read-only mode blocks its writes. Save report... writes the results as an HTML page like the commissioning report.

Sequences are saved as `.mbtest.json` files, usually next to the map, and `modbench-cli test` runs them too (Chapter 12).

10.12 Statistics

Every request sent on the Main page since the last Reset:

By result
How many succeeded, timed out, came back as each exception, or arrived as bad frames.

By function
Requests, failures, and minimum, average and maximum response time per function code.

A pattern points to the cause: timeouts to wiring, baud rate or unit ID; exception 2 to an address the device does not have; bad frames to noise or two devices with the same unit ID.

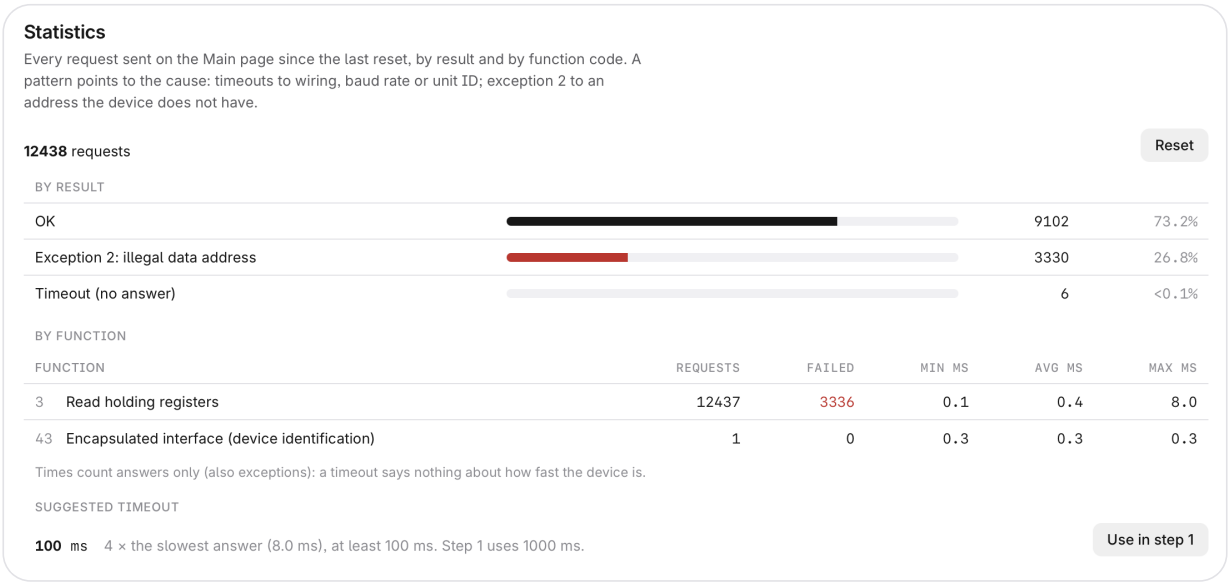


Figure 24. Statistics of a long test.

Suggested timeout is four times the slowest answer; **Use in step 1** puts it in the connection settings.

10.13 Alarms

Limits on the registers of the open map. A value below **Low** or above **High**, or a bit in its alarm state, turns red on the Device map tab.

1. Open a map on the Main page.
2. **Add an alarm on a register...** and choose the register.
3. Set the limits, or for a bit, the state that is an alarm.

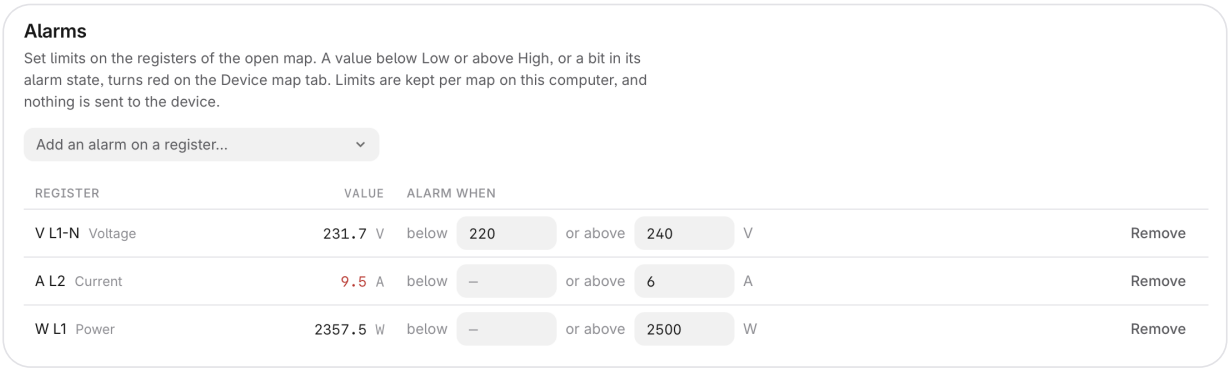


Figure 25. Alarms on a meter's map.

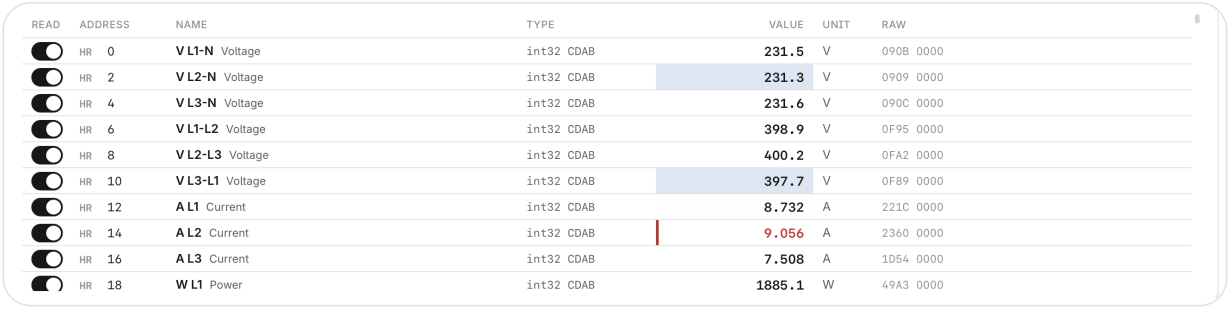


Figure 26. The value in alarm, in red on the Device map tab.

Alarms are kept per map on this computer. Nothing is sent to the device.

CHAPTER 11

Settings

Settings are kept on this computer and apply at once. The list on the left groups them as on the Tools page: *Modbench* (About, Updates), *Display* (Appearance, Values) and *Devices* (Connection, Device library).

11.1 About

The version of Modbench and whether it is up to date, with `Check for updates`. Below it are links to this manual, the release notes, a form to report a problem, the source code and the project's Buy Me a Coffee page.

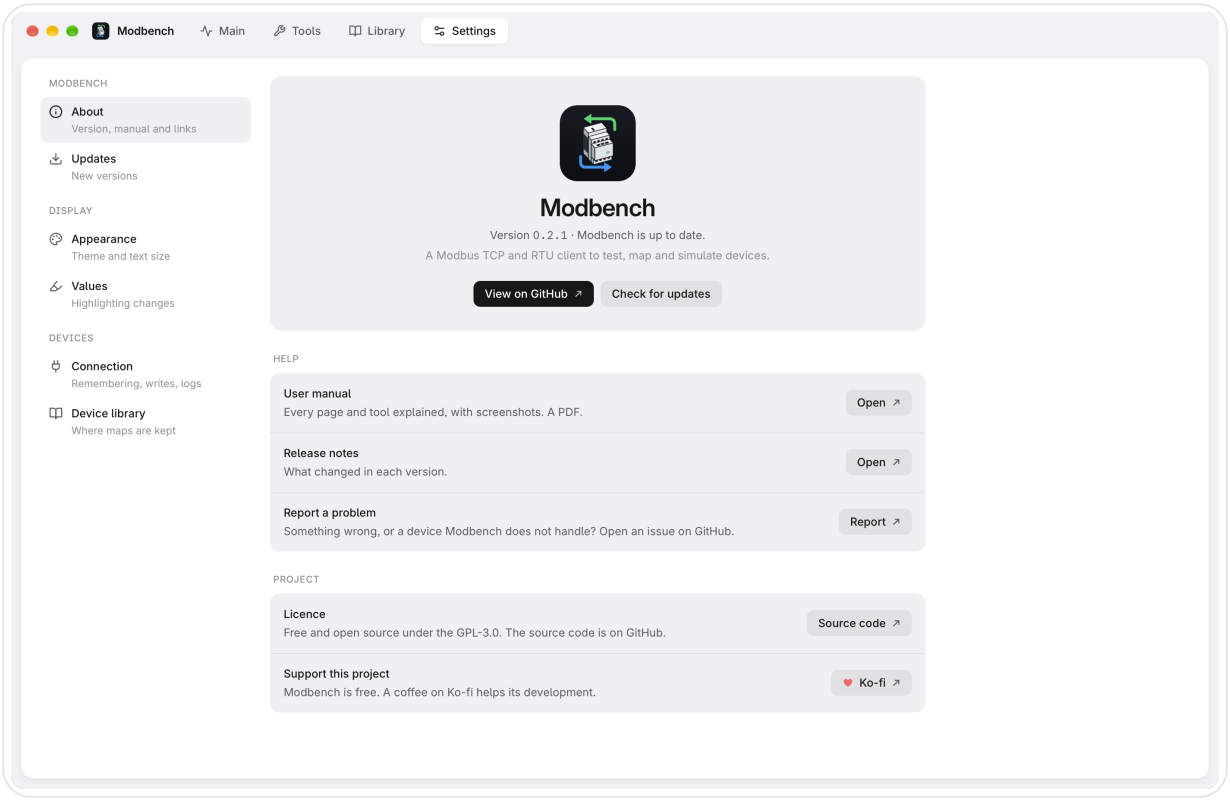


Figure 27. The About section.

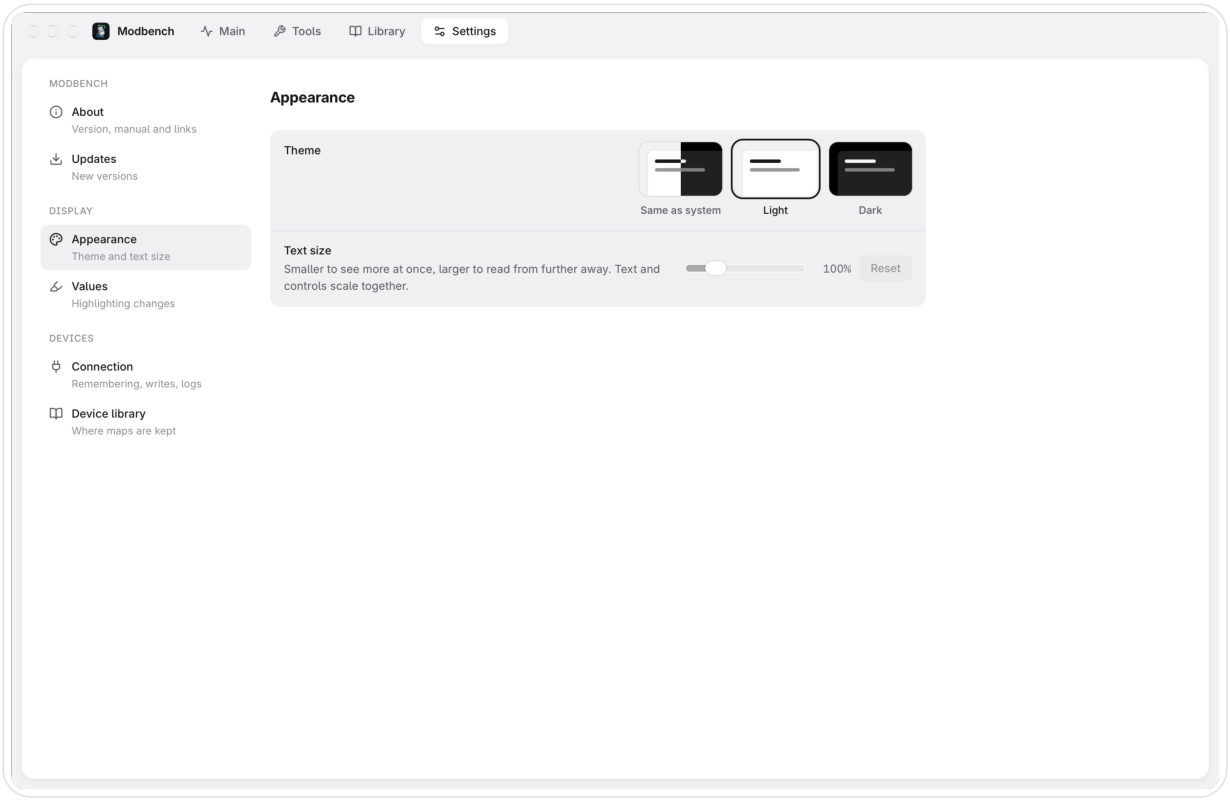


Figure 28. The Settings page, on Appearance.

11.2 Appearance

Theme
Light, dark, or as the system.

Text size
Smaller to see more at once, larger to read from further away. `Reset` goes back to 100 %.

11.3 Values

Highlight changed values
How long a value that changed stays highlighted: off, or 1 to 30 seconds.

11.4 Device library

Library folder
Where your maps are kept. `Open folder` shows it.

Shared maps
Links to the map format and to the repository of maps shared by other users.

11.5 Updates

Check for updates when Modbench starts
On by default. Installs from the APT repository update with `sudo apt upgrade`, and the Arch package with `pacman -U` (Section 2.3), instead.

11.6 Connection

Remember the last connection
Port, baud rate, address, device and timing are filled in again when Modbench starts.

Remember the connection of each map
Loading a map restores the port or host, baud rate, parity, stop bits and unit ID last used with it. On by default; without a stored connection the map's own defaults apply.

Remember register choices for each device
Which registers are read, the byte order and Loop come back for each device (map, connection and unit ID). Restoring them never writes or starts a test.

Forget saved device choices
Clears what was remembered per device.

Confirm before writing to a device
Asks before every write.

Reconnect when the connection is lost
While a test runs, a connection that fails is opened again, and the test goes on.

Read-only mode
Blocks every write, also on a connection that is already open.

Write single registers with FC16
One holding register is written with FC06 (Write Single Register) unless this is on; then FC16 (Write Multiple Registers) is used, for devices that only accept FC16.

CSV separator
For saved logs, exported values and recordings. Use a semicolon for spreadsheets set to a language with a decimal comma.

Requests kept in the log
Older requests are dropped first.

11.7 What Modbench keeps

Everything stays on this computer: settings, the last connection, choices per device, alarms, templates and the library folder. The only network requests besides your devices are downloads: the update check, the online library, and the vendors' icons from their websites.

CHAPTER 12

Command line and AI assistants

`modbench-cli` reads devices and works with maps without the window: for scripts, for servers without a screen, and for a quick check. It comes with the app. It never writes to a device.

12.1 Commands

```
modbench-cli ports
modbench-cli read --tcp 192.168.1.10 holding 0x0405 1
modbench-cli read --udp 192.168.1.10 holding 0 10
modbench-cli read --tcp 192.168.1.10 --source 192.168.1.5 holding 0 2
modbench-cli read --tcp 192.168.1.10 --tls --ca ca.pem --cert me.pem --key me.key
    holding 0 1
modbench-cli map --rtu COM3 --unit 18 device.mbmap.json --filter voltage
modbench-cli check --tcp 192.168.1.10 device.mbmap.json --report out.html
modbench-cli scan --rtu COM3 --baud 9600 --from 1 --to 32
modbench-cli scan-serial --rtu COM3 --first
modbench-cli id --tcp 192.168.1.10
modbench-cli simulate device.mbmap.json --strict
modbench-cli simulate device.mbmap.json --generators moving.json
modbench-cli simulate device.mbmap.json --rtu-over-tcp
modbench-cli simulate device.mbmap.json --rtu COM5 --baud 19200 --parity E
modbench-cli gateway --listen tcp:0.0.0.0:5502 --rtu COM3 --baud 9600
modbench-cli monitor --rtu COM3 --baud 19200 --parity E
modbench-cli test --tcp 192.168.1.10 device.mbmap.json start.mbtest.json --allow-
    writes --report out.html
modbench-cli validate device.mbmap.json
modbench-cli convert old-device.mbmap
modbench-cli export device.mbmap.json -f home-assistant --tcp 192.168.1.10
```

Command	What it does
`ports`	Lists the serial ports.
`read`	Reads coils or registers, like a manual request.
`map`	Reads every register of a map from a device, by name; `--filter` keeps the names that match.
`check`	Reads every register of a map once, as `map` does, and reports for each one whether it passed, failed or was not read, like the Check device button of the Device map tab (Section 4.4); it exits with 1 when a check fails and `--report` writes the HTML report.
`scan`	Finds the unit IDs that answer.
`scan-serial`	Finds the baud rate, parity and stop bits of a serial device: it asks one register (`--area` and `--address`) with each setting of `--bauds`, `--parities` and `--stop-bits` and lists the ones that get an answer; `--first` stops at the first.
`id`	Device identification (FC43).
`simulate`	A simulated device shaped after a map (port 5020 by default); `--strict` answers only the map's addresses; `--tls-cert` and `--tls-key` answer inside TLS, and `--tls-client-ca` asks clients for a certificate from that CA.
`gateway`	Passes requests from clients on `--listen` (`tcp:`, `udp:`, `rtu-over-tcp:`, `rtu-over-udp:` and a port, or `rtu:` and `ascii:` and a serial port) to the device of the connection options, and prints each one; `--unit` sends them all to one unit ID, `--read-only` refuses writes.
`monitor`	Listens to a serial line (`--rtu` or `--ascii` and its settings) without sending, and prints each request with its answer; `--json` prints one JSON object per request.
`test`	Runs a test sequence (`.mbtest.json`) against the device and exits with 1 when a step fails; `--report` writes the HTML report. A sequence that writes needs `--allow-writes`.
`validate`	Checks a map against the format, with the same messages as the app.
`convert`	Converts a version 1 map to a `.mbmap.json` next to it.
`export`	Writes a map as the configuration of other software (`-f home-assistant`, `esphome`, `telegraf`, `node-red` or `csv`); the connection options fill in the device's address.

With `--tcp`, `--tls` talks Modbus/TCP Security (port 802 unless the address gives one): `--ca` is the CA the device's certificate must be signed by (without it, any is trusted), `--cert` and `--key` the client certificate and key for devices that ask.

`read`, `map`, `check`, `scan`, `scan-serial` and `id` take `--json` to print JSON instead of text.

`modbench-cli --help` lists every command and option, and `modbench-cli --version` prints

the version; `-h` and `-V` are short for them. `modbench --help` and `modbench --version` answer the same way for the app, without opening its window. A wrong argument prints an error and exits with code 2.

12.2 Generators file

`--generators` takes a JSON list. Each entry names a map register (in engineering units) or a raw `area` and `address`, a `kind`, and how often it changes (`every_ms`, 1000 by default, at least 50):

```
[
  {"register": "Voltage L1", "kind": "sine", "offset": 230, "amplitude": 5, "
    period_s": 10, "every_ms": 500},
  {"area": "input", "address": 0, "kind": "increment", "step": 1, "min": 0, "max":
    100},
  {"area": "holding", "address": 1, "kind": "random", "min": 10, "max": 20},
  {"area": "discrete", "address": 0, "kind": "copy", "from": {"area": "coil", "
    address": 0}}
]
```

12.3 AI assistants (MCP)

`modbench-cli mcp` serves the same reads over the Model Context Protocol (stdio), so an AI assistant such as Claude Code can read devices and check maps: `list_serial_ports` lists the serial ports, `read_registers` reads coils or registers once, `read_device_id` asks a device to identify itself, `read_map` reads a device with a map and `validate_map` checks a map file. Like the command line, it never writes to a device.

Letting an assistant drive the window

The tools above run in their own process. To let an assistant work in the Modbench window you are looking at, turn on *Let an assistant control Modbench* in Settings, Connection. It is off by default. Modbench then listens on a loopback port only, and writes the port and a random token to a file in its data folder; `modbench-cli mcp` reads that file and forwards the `gui_*` tools to the window. Without a running window, or with the setting off, those tools say so and the others keep working.

gui_get_state
The visible state as JSON: connection, open maps, registers and their latest values, the screen shown. Read it first.

gui_set_connection
Transport, host or serial port, port, baud rate, parity, stop bits, unit ID, timeout.

gui_list_library, gui_open_map
Browse the library and open a map.

gui_set_map_view
Address notation, byte order, which registers are read.

gui_set_manual_read
Function code 1 to 4, address and quantity.

gui_run
Start or stop reading, with Loop and its interval.

gui_chart
The registers on the Chart tab.

gui_navigate
The page, tab or tool shown.

What you can rely on:

- Every change happens in the window, so you see it as it is made, and a bar shows that an assistant is connected, with a list of what it did.
- *Pause* turns every request down until you resume it. *Disconnect* drops the assistants and changes the port and token.
- While a test runs, the connection, the map and the manual read cannot be changed; stop the test first.
- It never writes to a device: manual requests are limited to the read function codes, and a manual request that is a write is not started. Read-only mode and the write confirmation are therefore not needed for it, and stay as they are for you.
- Only programs running as your user can read the token file.

APPENDIX A

Troubleshooting

A.1 No answer (timeouts)

On a serial line
Check, in this order: the unit ID; baud rate, parity and stop bits (Find... tries them, Section 3.5); the A and B wires, which are often swapped; the termination at both ends of a long line; and that no other master talks on the same line.

On the network
Ping the address; check the port (502, or the converter's); some devices accept one client at a time, so close other programs connected to it. Through a gateway, the unit ID selects the device behind it.

Slow devices
Raise **Timeout**, and add a **Delay** between requests. Statistics suggests a timeout from the measured answers (Section 10.12).

A.2 Values look wrong

Off by one register
The manual counts from 1 and the map, or your request, from 0, or the other way round. Try the address one lower or higher, or check `address_base` in the map.

Huge or tiny numbers
Usually the byte or word order. Try the **Byte order** override in step 2, or paste the registers into the Data decoder (Section 10.8).

Wrong by a factor of 10, 100...
The scale. Maps give it per register; some devices also have a scale factor register.

A mismatch banner
The device is not the one the map is for, or its firmware differs (Chapter 4).

A.3 Bad frames

A CRC or LRC error means the bytes arrived damaged: noise on the line, a missing termination, a wrong baud rate, or two devices answering at once because they share a unit ID. The Raw data tab shows the frame (Section 7.1).

A.4 Exception codes

Code	Name	Usually means
01	Illegal function	The device does not support this function code.
02	Illegal data address	The address, or part of the range, does not exist. Read fewer registers, or check the address base.
03	Illegal data value	A value or a quantity the device refuses.
04	Server device failure	The device failed to do it.
05	Acknowledge	Accepted; the result comes later.
06	Server device busy	Try again later.
08	Memory parity error	The device's memory failed a check.
0A	Gateway path unavailable	The gateway has no route to that unit ID.
0B	Gateway target failed to respond	The device behind the gateway did not answer.

APPENDIX B

Reference

B.1 Function codes

Code	Function	Most per request
01	Read coils	2000 bits
02	Read discrete inputs	2000 bits
03	Read holding registers	125 registers
04	Read input registers	125 registers
05	Write single coil	1 bit
06	Write single register	1 register
15	Write multiple coils	1968 bits
16	Write multiple registers	123 registers
43/14	Read device identification	—

B.2 Byte and word order

How the 32-bit value 0x0A0B0C0D arrives, in the four orders:

Byte order	Word order	On the wire	Common name
big	big	0A 0B 0C 0D	Modbus standard (ABCD)
big	little	0C 0D 0A 0B	Word swap (CDAB), used by many meters and PLCs
little	big	0B 0A 0D 0C	Byte swap (BADC), rare
little	little	0D 0C 0B 0A	Little-endian (DCBA)

B.3 Files

`.mbmap.json`
A device map, version 2: one JSON object with the device's connection defaults and its registers. The format is described in `SPEC.md` of the mbmap-library repository.

`.mbmap`
A device map, version 1 (older, line-based). Modbench opens it and converts it to version 2.

`.mbsim`
A simulator setup (JSON): the interface settings and each device's unit ID, map (with a copy of it), *Only the map's addresses* and generators.

Recordings (`.csv`)
Made with `Record…`: a time column, then one column per register, named with its unit; one row per cycle. The Trend player plays them back.

Logs and exports (`.csv`)
From `Save log…` and `Export…`, with the separator set in Settings.

B.4 Keyboard

Key	Does
`Ctrl+K`	Opens the command menu (`Cmd+K` on a Mac).
`Esc`	Closes a menu or a dialog; in the online library, goes back to the vendors.
`↑` `↓` `Enter`	Move through and choose in menus and lists.
`Tab`	Moves to the next field.